

Design & evaluation of a Hybrid Energy-harvesting System for Wearable Health & Environmental Monitoring

An Honors Thesis

Samanvay Upadhyay

May 2026

Professor Jeremy Gummeson, Faculty Sponsor

Electrical and Computer Engineering Department

Professor Yeonsik Noh, Secondary Reader

Electrical and Computer Engineering Department

Abstract

In this thesis, we analyse a hybrid energy harvesting system for wearable sensing, which consists of a photovoltaic, thermoelectric, and kinetic harvesters. The key contribution of this thesis will be in the hardware-grounded system analysis: measured traces are used to describe the properties of each harvester for a set of realistic scenarios and then to test a scheduler under the hybrid storage framework. Under outdoor conditions, it turns out that the photovoltaic harvesting path generates the maximum average output, while the thermoelectric one acts as a baseline source of energy, and the kinetic harvesting path produces its maximum output during feet movement. As a result, the proposed solution can be best understood as a complementary multi-source energy system, rather than three substitutable harvesters. According to scheduler analysis results, energy-aware strategies allow sustaining a periodical sensing process and sending reports when the harvesting traces are favourable; meanwhile, more conservative policies tend to decrease the number of measurements when there is less energy available from the sources.

Introduction

This honors thesis explores the design, simulation and scheduling of a hybrid energy-harvesting system meant for battery-less wearable sensing. The system utilizes three ambient energy sources (photovoltaic, thermoelectric and kinetic) and uses energy-aware sensing and transmission scheduling to allow perpetual battery-less operation with a small energy buffer (small battery & supercapacitor). Three hardware validated simulation modules (SEH: PV model and IV fitting, TEH: transient/steady two-node thermoelectric solver with ambient weather matching, KEH: transient piezo models with rectifier + capacitor simulation) along with a scheduler that uses per-session outputs from those modules have been created for this project in my 499TP honors research. These simulation modules initially used publicly available datasets for these energy harvesters to build representative combined timebases for realistic use scenarios (outdoor exercise, indoor movement, rest), which were then verified with a hardware prototype consisting of PRMT07-18305-42 for TEH, Adafruit 5 V / 0.6 W mini solar panel for SEH and S129-H5FR-1803YB for KEH to estimate achievable sensing uptime and data quality and maximize useful sensing and transmission while minimizing energy buffer size by exploring scheduling policies.

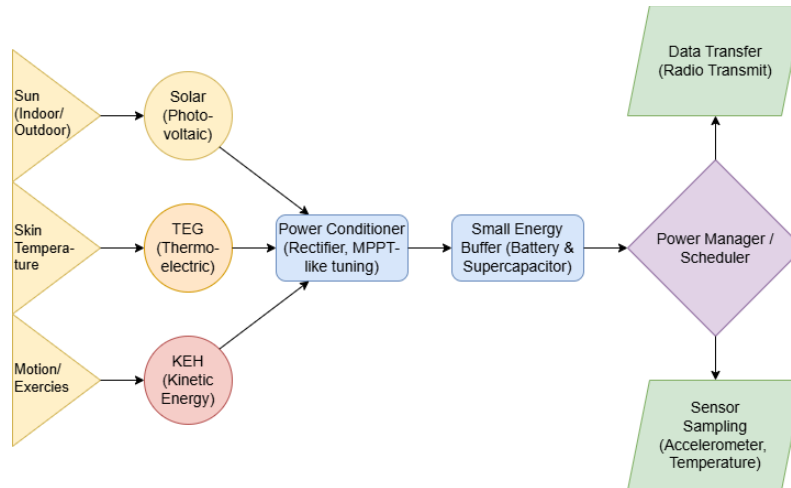


Figure 1: General overview of the Hybrid Energy-Harvester's system architecture setup.

Wearable health monitors are ubiquitous but constrained by batteries that affect the device's form factor, lifetime and environmental sustainability. Hybrid energy-harvesting intends to reduce, or even remove, this reliance on batteries by capturing power from multiple sources of ambient energy. However, creating a practical battery-less wearable device requires co-design between harvesters, power electronics and task scheduling, especially policies tailored to the intermittent and multi-modal nature of energy harvesting patterns in the real world. This research uses simulation and scheduling models at the system level to measure how well hybrid solutions perform, along with their limits, for real-time health monitoring and general sensing tasks.

Scientific Question

Is sustained periodic sensing and intermittent wireless reporting for a sensing system possible with a hybrid energy-harvesting system (solar + thermoelectric + kinetic) and to what extent? What scheduling strategies minimize energy storage requirements while maximizing the sensing uptime and data quality across realistic active and resting scenarios.

Objectives.

1. **Measure energy supply** from PV, TEG and KEH under realistic usage scenarios using the three simulation models. Generate stats per session (mean power, energy per session, duty factors, power quantiles) and compute resampled distributions across each scenario.
2. **Develop and test scheduling policies** (periodic, threshold, opportunistic, predictive) to maximize sensing uptime and data quality while lowering energy storage costs.
3. **Understand storage sizing tradeoffs** by determining the minimum energy buffer size to meet performance goals (sampling quality, transmission reliability) under each scenario.
4. **Produce a tested simulation based design framework** for a battery-free wearable prototype such as expected sampling uptime outdoors and indoors, recommended buffer capacity and harvester sizes.

Deliverables.

1. Per-session statistics for PV, TEG and KEH datasets.
2. Scenario-based combined timeseries and bootstrap distributions.
3. Comparative scheduler evaluation based on performance and energy storage metrics.
4. Thesis manuscript, final presentation and hardware prototype of the hybrid energy-harvesting system.

Significance and relationship to literature

The literature review covers hybrid energy harvesters along with software/hardware frameworks necessary for batteryless devices. Recent reviews point to hybridization of energy harvesters as a way to reduce needed energy storage while improving dependability [4][5]; recent prototypes also show integrated harvest and storage and batteryless wearables [6][8]. My project advances this field by combining high-fidelity simulation models for the three harvester types and scheduler simulation to understand system-level tradeoffs. The area of my work remains

under-quantified in the literature because prototype measurements are costly and environment dependent.

Importance to knowledge advancement.

- 1. System-level insight:** Instead of one device, the research looks at multiple energy inputs and how they interact with energy-aware scheduling. This provides practical guidance to designers of battery-less wearables on sizing and policy choices.
- 2. Methodological contribution:** The simulation and scheduler combined approach produces reproducible results of scenario-specific estimates for storage sizing and uptime which enables faster design iterations without testing on hardware first.
- 3. Application relevance:** Results from this research will directly inform the feasibility of battery-less monitoring under typical real-world scenarios using hybrid energy-harvesting. It will also help huge practical harvesters and scheduler pairings.

Literature Review

Introduction

Wearable tech—fitness bands, smart glasses, and biomedical sensors—is increasingly being used owing to improvements made possible by flexible sensor technologies and microcircuit integration[1]. However, energy supply is a prime challenge for such devices. Traditional energy supply solutions tend to be hard and bulkier, thus limiting the overall usability and device design. Moreover, they tend to degrade with time and thus require periodic recharging and replacement[1][2]. The possibility of replacing them on a periodic basis for devices meant for use at a distance and for on-body use may be difficult and raise concerns for the

environment[3][4]. To overcome these difficulties, various harvesting solutions for extracting ambient energy from various heat and chemical processes and for powering the device seamlessly have begun to emerge[5][6]. Hybrid energy harvesters that collect energy from various sources tend to be more efficient and reliable. Most current literature in this field still focuses on single harvester sources like piezo or light or lacks the long-term, multi-modal datasets that my research will be generating. Another recent advancement is batteryless Internet-of-Things solutions that use small capacitors or supercapacitor storage for power and do not interrupt services even with varying energy supply through harvested energy[2][6][7][8]. This review provides insight into the advances from 2019 to 2025 for wearables with energy harvesting and batteryless design. These wearables function on the premise of power-harvesting methods instead of battery power and thus tend to be highly dependent on accessible ambient energy; therefore, they demonstrate highly dynamic behavior for task execution. Due to the constantly changing power conditions, the task scheduler needs to adapt at a highly accelerated rate for uninterrupted functioning of basic operations. Some wearables tend to forecast tasks for high-priority accomplishment when resources are limited. Energy harvesters for wearables harness the energy from motion, heat, and light and transform them into electrical energy for sensors and health monitors. A significant number of emerging wearables allow uninterrupted operations without the requirement for recharging from outside energy resources. Research from the literature suggests advancements mainly from experimentation-based approaches and solutions instead of proposals at the concept level and from recent publications within the current decade. A key factor missing from papers and physical prototypes in this field is the simulation-first approach present in my work that allows for testing corner cases (e.g., long periods of indoor inactivity).

Hybrid Energy Harvesting

Hybrid energy harvesting combines various energy conversion processes and leverages the environment for simultaneous harnessing of various energy forms. The combination and integration of various energy types, such as solar and mechanical energies, increase overall energy and stabilize energy supply and flow rates [9][10]. Liu et al. describe the core aspects and elements of design for Hybrid Energy Harvesting (HEH) and cover aspects such as material properties and design aspects, structural design and integration approaches, and application areas and analysis for HEH and energy harvest integration and applications for various products and design types [11]. It is observed in various literature works that overall efficiency can be increased with simultaneous use and conversion and harnessing from multiple energy types and resources and combination and integration approaches such as joint solar and piezoelectric energy harnessing as well as joint harnessing and conversion from solar and thermoelectric energy types and resources and methods and approaches [9][11]. For example, harnessing and conversion from solar energy with thermoelectric and kinetic energy originating from various motion activities and events would increase overall net energy harvesting among various lighting and motion activities. Kang and Yeo point to specific hybrid implementations: Ren et. al made a flexible device that links an organic solar cell to a single-electrode TENG, improving the harvester's voltage and current by ~120% and 105% with respect to having them as individual devices[12][13]. Similarly, Zhao et al. showed a hybrid nanogenerator that captures piezoelectric (mechanical) and solar energy, delivering greater power density than standalone transducers [12]. The advantages of combined harvesting methods are starting to be measured more precisely. Simone et al. used actual climate records like sunlight and wind to analyse hybrid harvesting. Their findings indicate that well-structured integrated systems keep running steadily despite

changing environments and need far less stored power. For example, their results “show that one of the main advantages of the hybrid solution is the reduction in the size of the storage device, enabling the replacement of rechargeable batteries with supercapacitors”[10][14]. In simple terms, hybrid energy systems may reduce the need for large buffers, making it easier to run devices without batteries. Liu et al. noted that synergetic hybrids like piezoelectric–pyroelectric films or tribo–photoelectric stacks can deliver high power output per area while maintaining flexibility [3].

Designing hybrid harvesters needs precise setup. Different transducers should match both electrically and mechanically and power-management circuits must handle multiple inputs[9]. Kang and Yeo highlight difficulties in handling power from various origins, pointing out that control logic is essential to match varying voltages or currents while also improving energy transfer efficiency[2][9]. Still, more studies now describe hybrid setups: Recent wearable prototypes have been combining organic photovoltaics with piezo/tribo generators or with thermoelectrics[6][9]. With better materials and flexible electronics, hybrids offer better power and dependability: pairing body-heat TEGs with small PV cells can supply better stable current both day and night; vibration harvesters can contribute additional power during peak motion periods. Overall, combined or multiple-energy sources are becoming a reliable way to increase usable power and maintain functionality in wearables[13][10].

Batteryless Sensor Systems and Intermittent Computing

Batteryless sensor nodes rely only on collected energy, using capacitors or tiny supercapacitors for short-term storage instead. To function, these transiently powered systems gather sufficient charge before activating for sensing or sending data. Consequently, such devices work in bursts: charging first, running briefly, then turning off until more energy is gathered[7]. Unlike

conventional battery-driven units with steady power supply, their operation isn't continuous but cycles with available energy. Mosavat et al. point out that without batteries, devices usually run in short bursts because stored power is limited; as a result, steady performance becomes difficult compared to systems using batteries[7]. In line with this, Ahmed et al. argue that scaling to more IoT units requires designs that work reliably while avoiding both upkeep and reliance on traditional power sources [3][4].

When devices run on and off, it creates challenges for consistent operation. During outages, preserving the processor's condition becomes critical. Engineers apply nonvolatile storage like FRAM to save data before shutdowns. Alternatively, they organize code into repeat-safe blocks that handle resets without errors. Lukas et al. developed a working battery-free package system meant for wearable tech[2]. Their prototype includes an ultra-low-power SoC, FRAM (non-volatile boot memory), and a radio; it can sense and process data (at only $\sim 1.02 \mu\text{W}$ average power) and recover from power loss without losing state[2][8]. Other works design software frameworks and compilers (e.g. Alpaca, Chain, Mementos) specifically to support intermittent execution. Ahmed et al. note that the community has indeed developed new energy-efficient programming languages, compilers, runtimes, and architectural mechanisms to enable practical batteryless IoT applications[3]. These advances include checkpointing libraries, loop invariants, and task scheduling mechanisms tailored to the on/off power pattern.

In networked sensing, running without batteries impacts communication. For example, Mosavat et al. explore ways to support wireless links between energy-limited, battery-free devices[7].

Since standard radios use too much power even while waiting, they suggest using wake-up receivers (WuRs), which monitor channels more efficiently. This reveals an essential design trade-off: adding functionality such as connectivity while respecting irregular, scarce energy

supplies. As a result, batteryless wearables typically carry out basic actions intermittently, capture small data amounts, send them quickly, then enter low-power mode, timed according to available energy.

On the whole, moving to batteryless sensors means changing both hardware and software approaches. While microcontrollers need to run steadily on tiny amounts of power, usually through energy-harvesting circuits, peripherals should use as little energy as possible. Systems with fewer components, like direct wireless links or event-triggered data collection, work better here. Even so, there are working models of entirely battery-free wearables: one uses a flexible patch powered by sunlight[6], another runs a fabric-based pressure grid via a slim thermoelectric layer[2]. Altogether, smart engineering shows such devices can function well when active only briefly and occasionally.

Energy-Aware Scheduling in Batteryless Devices

Due to limited, unpredictable power, timing and resource handling matter greatly in devices without batteries. Tasks like sensing or sending data need alignment with available energy supply. In real use, smart scheduling appears in various ways. At minimum, a device tries an action once its internal charge reaches a threshold. Better methods assign fixed times or rank jobs by expected energy levels alongside how urgent they are.

Recent studies looked at timing methods for devices using harvested energy. Sabovic et al. showed a system that schedules jobs based on available power, considering both task links and incoming energy flow [3]. While approaches differ, they mainly aim to prevent situations where big tasks stop halfway due to low power. For example, the system could split heavy computations into chunks matching regular charging periods. Alternatively, it may postpone less urgent actions until more energy arrives, like sunlight powering solar sensors.

In networked settings, managing power use ties into how messages are sent. According to Mosavat et al., deciding to use wake-up radios counts as timing control: rather than staying active nonstop (an inefficient approach), the device remains in deep sleep until a separate WuR trigger activates it briefly[7]. Other protocols similarly incorporate knowledge of each node's energy state. The goal is to maximize forward progress, the total useful work done (sensing or transmitting data) while avoiding wasted attempts when a device is energy-starved.

These approaches follow a simple rule: use no more energy than what is gathered over time. For example, in battery-free IoT systems using tinyML, tasks might run locally or be sent elsewhere depending on available power at the moment. Instead of frequent writes, smart caching helps save resources when electricity is scarce - recent studies point this out. According to Ahmed et al., progress continues here; researchers are building easier tools for developers, such as code environments and timing controls that adjust themselves if power drops suddenly[2].

In conclusion, energy-aware scheduling in batteryless wearables involves dynamically prioritizing or fragmenting tasks to fit within available energy, leveraging low-power hardware features to minimize idle losses and co-designing system software to checkpoint and recover state. These approaches together help ensure that batteryless wearables can perform their sensing and communication tasks reliably even under unpredictable power input.

Wearable Harvesters and Applications

A variety of energy-harvesting devices have appeared on wearables. Sunlight is easily available outdoors, making solar cells common. Thin organic and perovskite solar units are now built into flexible tech or clothes. Jinno et al. developed soft, waterproof organic solar films for smart textiles[6], working well even when bent around body shapes. Some tiny fiber-shaped solar elements are already woven directly into cloth structures [7].

Thermoelectric devices convert body heat into electrical energy by applying the Seebeck effect. Since skin runs at roughly 34°C and ambient air sits near 22°C, just a narrow thermal gradient emerges - so power levels usually stay in the microwatt range per cm². Still, flexible TEG units work well in wearable formats like wristbands or adhesive patches. Wang's team in 2020 linked ultra-thin thermoelectric layers with sensors to make garments that sense touch while generating their own current[10]. As long as there's a temperature contrast, these units deliver consistent direct current, unlike motion-driven systems prone to fluctuations.

Mechanical harvesters using piezoelectric, triboelectric, or electromagnetic principles capture energy from movement. When deformed, materials such as PVDF generate electric charge through piezoelectric effect. TENGs create power via surface contact and separation, often using fabric layers that exchange electrons. Zhao et al. built a hybrid nanogenerator combining piezoelectric and photovoltaic elements to scavenge light and motion[12]. Piezo/tribo generators can output pulses in the 10–100 µW range during active motion (e.g. walking or bending), but generate no energy at rest which is why they are extremely relevant in hybrid designs.

Integrated harvesters and energy storage combine the above methods. For example, Sun et al. wove a textile “solar-mechanical” cable that simultaneously generates electricity from light (organic PV) and motion (triboelectric layers)[4]. Such multifunctional fabrics can power small sensors continuously in real-life use. These smart harvesters provide steady power to tiny sensors during everyday activities. Because they capture energy in multiple ways, such materials support various body-worn devices. While individual designs suit specific scenarios, mixed approaches should deliver steadier output for upcoming wearable technologies.

Methods

The research is divided into a two-stage methodology:

1. **Simulation stage (completed and reproducible):** generate modality and timestamp-based harvested power traces using PV cells, TEG and KEH. Align and combine the traces into representative scenario timeseries, understand power generation based on these scenarios and evaluate scheduler policies on an energy-buffer model. These steps were implemented in the provided python codebase and produced bootstrap numbers and sampling-rate feasibility estimates used in this proposal.
2. **Harvester prototype stage (completed):** implement lab bench tests and wearable prototype experiments to compare actual harvester outputs to simulation results and measure actual per-event energy costs on a target hardware platform. Refine the simulation models based on hardware results. The harvesters utilized were PRMT07-18305-42 for TEH, Adafruit 5 V / 0.6 W mini solar panel for SEH and S129-H5FR-1803YB for KEH. Hardware prototype also allowed for more refinement in terms of understanding heatsink simulation for TEG, Harvester- shoulder angle simulation for SEH., and tip-mass and resonant frequency simulation for KEH along with general harvester placement.

System Overview and Hardware Description

The photovoltaic path uses a small ETFE solar cell (rated at 0.6 W and 6 V). It has an open-circuit voltage of 7.09 V, while its maximum power voltage is 6.07 V. The thermoelectric path is modeled using the Prometheus-class energy-harvesting module. Its cold start temperature

is 0.5°C , and the thermal impedance matches the heatsink value used for the modeling system.

The kinetic subsystem uses the shoe-mounted piezoelectric harvester with a resonant frequency of 175 Hz. The tip-mass sweep approach is used in choosing the proof mass to maximize harvestable power.

The storage subsystem of the energy management prototype is built around the hybrid storage approach. In this case, the model uses a supercapacitor alongside the battery storage mechanism. The sampling schedule ensures that energy remains usable in the storage subsystem, thus ensuring optimal sampling, transmission, and operation holds. The choice of architecture is fairly standard, but it is necessary. Without it, the harvesting devices cannot make the system function as intended. In essence, the storage subsystem translates ambient energy into workable energy for the node.

Simulation models

1. PV pipeline

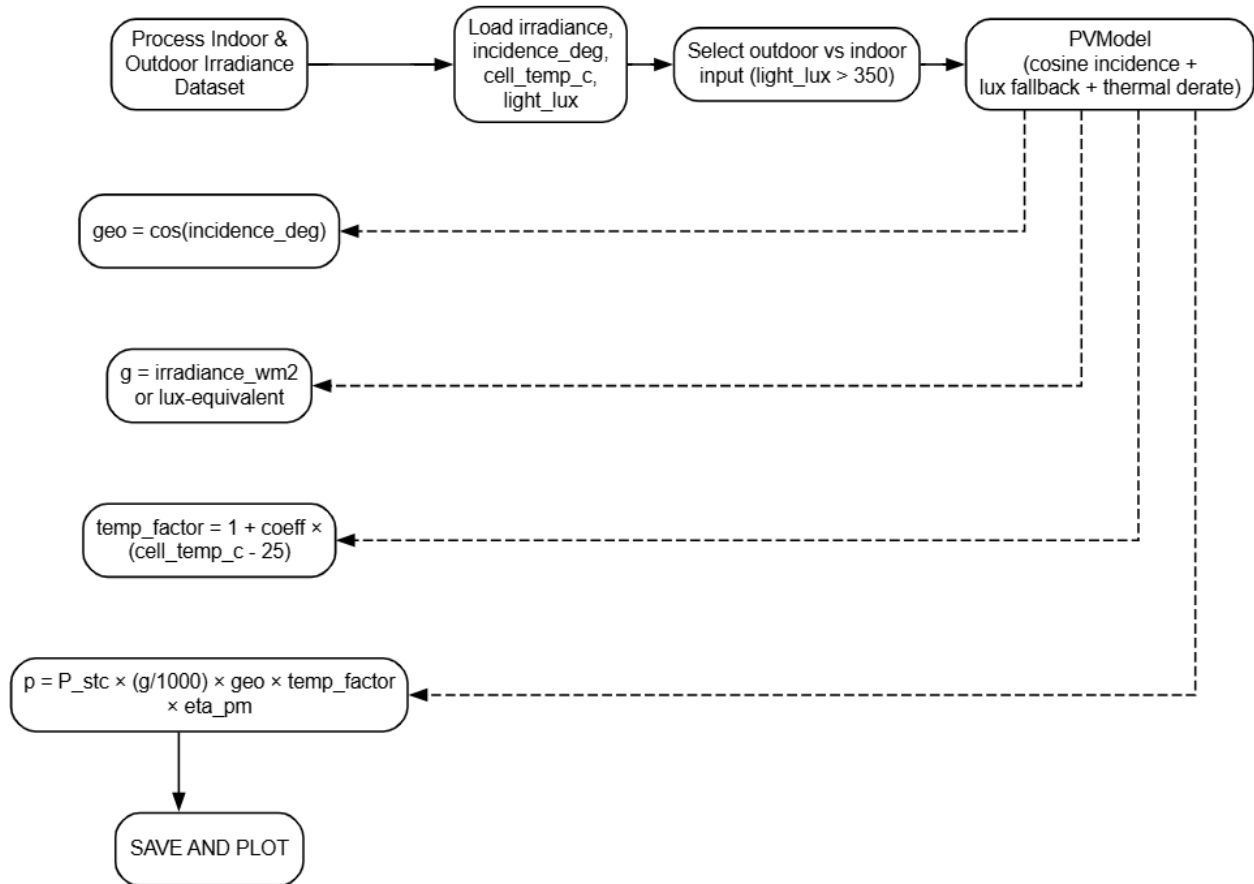


Figure 2: Simulation Model for Photovoltaic cells

The dataset used for the PV cell comes directly from the hardware prototype where the PV cell is worn on the shoulder with a magnetic clip. The simulation model confirms the data by mapping the computed power to an Implied G to detect implausible fit results.

- a. Input .csv filetype datasets of irradiance and V/I sweeps, convert lux to W/m^2 heuristic, NOCT temperature estimate, single-diode least-squares fit and datasheet fallback.
- b. Output per sample $P_{\text{model_W}}$ and $P_{\text{elec_W}}$.

Photovoltaics must be classified as the major high-power mode in outdoor environment.

While the datasheet for the PV cell contains an estimate for panel size and

maximum-power point coordinates, the simulation code includes a simple yet clear formula for converting irradiance to power. Such formula depends on incidence angle, environmental conditions, and the backup indoor condition. The power generated by solar energy is high in case of illumination and decreases rapidly in shadow or indoor conditions.

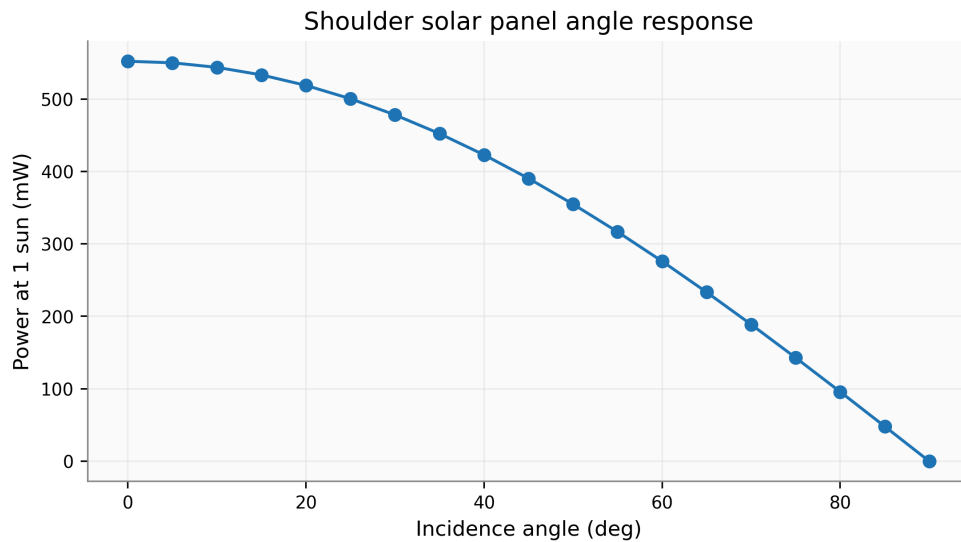


Figure 3: Photovoltaic response versus incidence angle.

This is a critical graph because it explains why the sun's trajectory cannot be treated as a source of constant power. The change in orientation and motion of the body while wearing a device changes the angle of the panel, which affects the amount of power generated. This graph was created during mid-April daylight in Western Massachusetts.

2. TEG pipeline

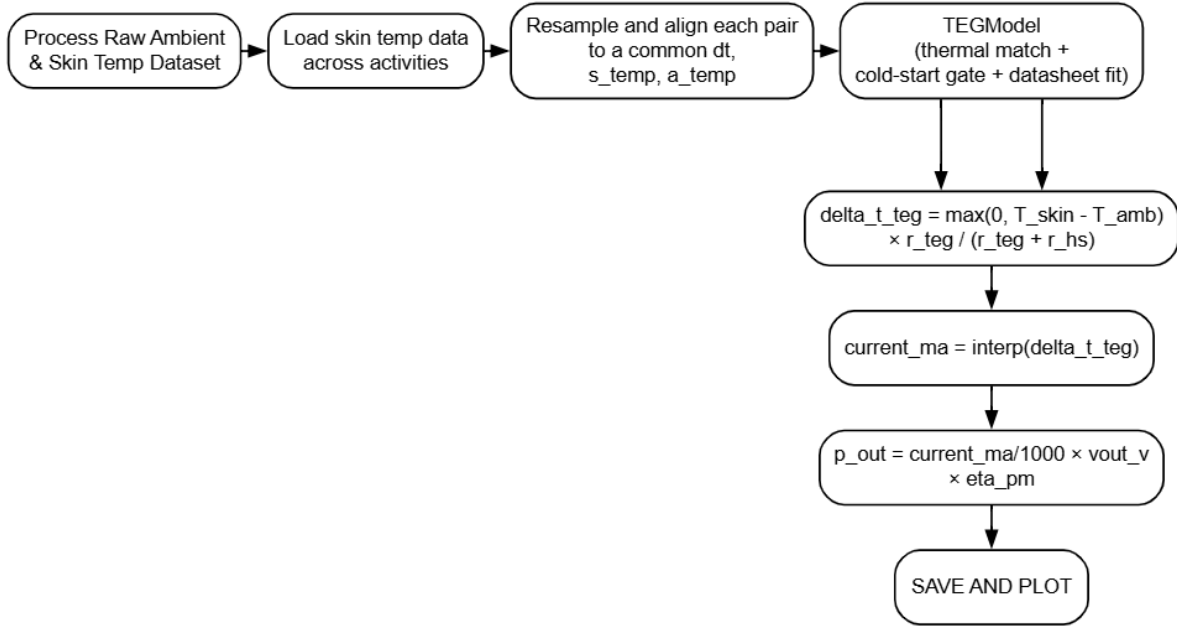


Figure 4: Simulation Model for Thermo-electric generators

The dataset used for thermo-electric generators is again, validated with the hardware prototype. The dataset provides meaningful insight into body temperature across various activities like walking and resting and exercising. The simulation code uses a two-node thermoelectric model for both steady state and transient solvers. It uses wearable CSV sessions from the dataset for ambient and skin temperature values. The code maps a module-level thermal resistance to the simulation area, calculates per-area convective and radiative coefficients and converts the legacy contact / convective resistance inputs to per-area conductances. The steady-state function deals with a nonlinear two-equation problem at every time step (concerning the temperatures of hot and cold nodes). The transient function `teg_transient_df` generates interpolators for the skin and ambient temperatures, initializes with the steady state, solves the ordinary differential equations for T_h and T_c with `solve_ivp`, and finally transforms the obtained solutions back into the original temporal

order.

- a. Input: wearable skin and ambient temperature
- b. Output: per-sample P_{load_W} , T_{h_C} , T_{c_C} and energy using two-node transient and steady thermal models.

Thermoelectric generation forms the most reliable energy source in the entire system. In the simulation model based on the hardware prototype, there is a slight cold start threshold, along with a thermal network, where the heatsink resistor forms part of the control equation. As a result, the output is steady at sub-milliwatt in almost all the test cases. The basic conclusion drawn is that thermoelectric generators should form the base of the energy budget.

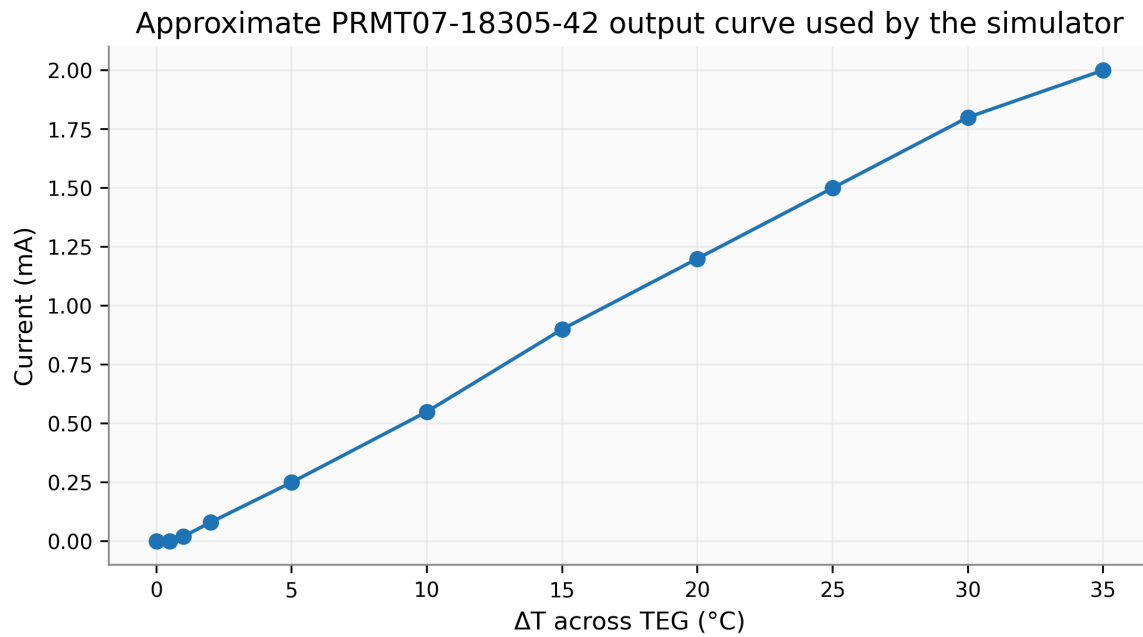


Figure 5: Thermoelectric module output curve

The corresponding graph confirms the point above that TEGs are steady. With respect to temperature difference, it has a steady rising curve, while the output value stays relatively

low when compared to solar generation with high light intensity. However, this trait is useful since the energy source is active during resting moments.

3. KEH pipeline

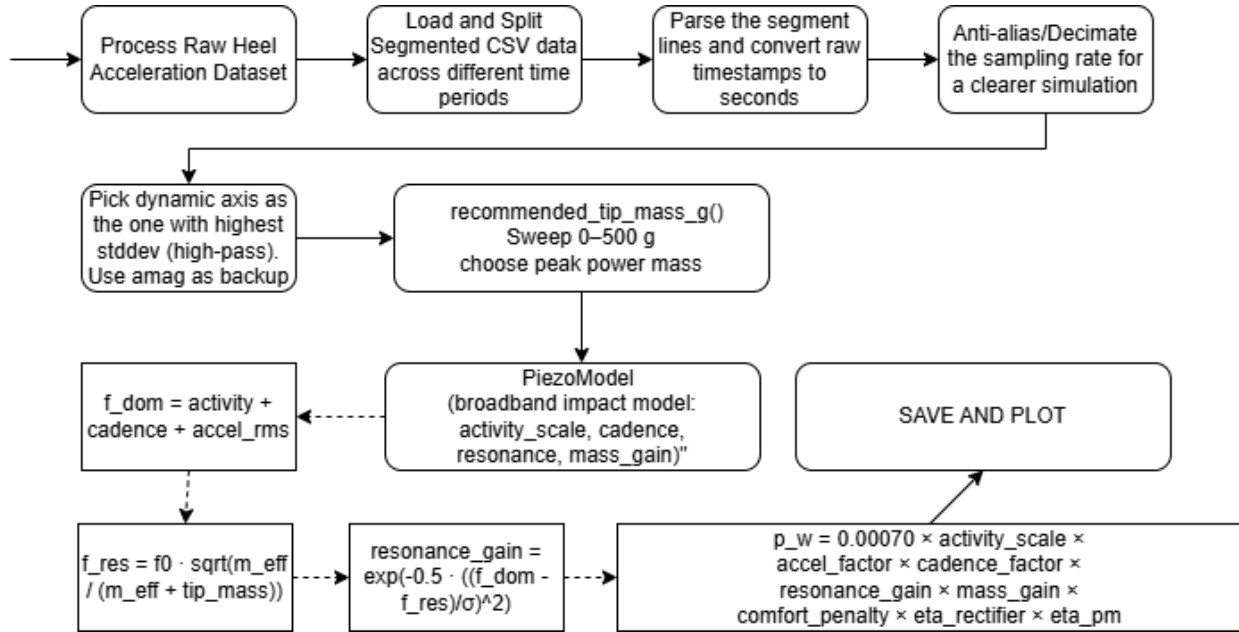


Figure 6: Simulation Model for Kinetic Energy-Harvesters

The dataset used for KEH simulation consists of heel striking data across various physical activities with the hardware prototype. The script KEH takes raw acceleration logs, transforming them into rectified harvester power traces, modeled with physics-based transient solutions for piezoelectric harvesters. The script reads timestamped acceleration logs, splitting them into monotonic chunks, which are then optionally resampled at a specified rate. The script also optionally selects a dynamical axis, high-pass filtering all three axes to remove the effects of gravity, then picking the axis with the highest dynamical variance. A coupled mechanical-electrical state with piezoelectric capacitance is modeled with a solution in PiezoHarvester. Following the solution of the transients, a stable simulation for a diode bridge with a capacitor

(diode_bridge_cap_simulate) is run to derive V_{cap} , I_{load} , and rectified P_{load} for specified charging resistance and capacitance.

- a. Input: Raw heel striking accelerometer logs
- b. Output: P_{load} parquets using transient piezo solver and diode-bridge & capacitor simulation produce rectified traces.

This kinetic path can be explained in terms of a shoe-based piezoelectric system that has a built-in resonance sweep. The crucial element in the hardware is the proof mass since the change in this parameter changes the frequency of resonance and thus the sensitivity of the transducer to motion. The code demonstrates this correlation by choosing a specific hardware instead of treating the kinetic harvester as an abstract motion source.

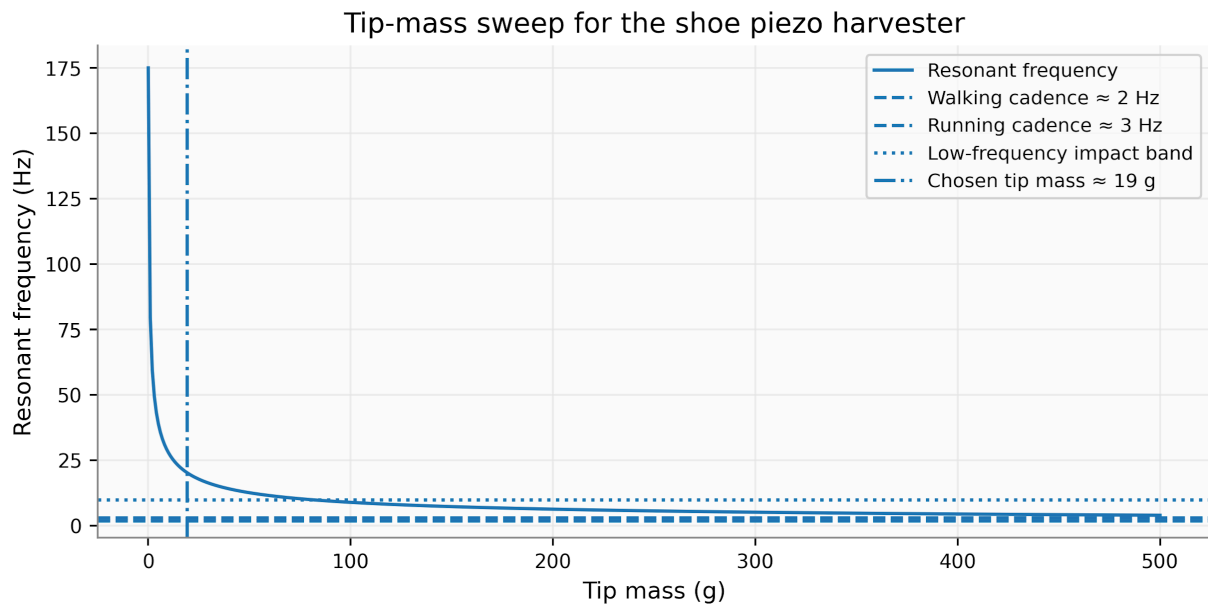


Figure 7: Tip-mass sweep for the shoe piezo harvester.

The selected tip mass is about 19 grams. This selection is based on the presence of a clear local maximum in the power curve as shown by the resonance sweep.

4. Power Management and Controller

To create the dataset from the harvesters, we also utilized a Power Management IC and a Microcontroller in the hardware prototype. The BQ25570EVM-206 is designed for energy harvesting with ultra-low power characteristics and therefore fits the purposes of solar, thermal, and piezoelectric harvesters. In this regard, it offers a fairly good basis for managing and regulating the energy coming in from different harvesters. It is important to note that different harvesters supply different amounts of voltage which is also managed by the buck converters present on the evaluation board.

The controller aspect of the proposed prototype is an MSP430FR5994 LaunchPad, which suits low power sensing applications as well as a portable storage-backed wearable controller. It is important to note that this aspect of energy management plays a vital role since it ties the whole process together, namely: harvesters define power availability, storage defines time limits of allowable inactivity, and scheduling decides when actions should be taken.

Measurement Methodology and Scenario Construction

The file for each scenario contains 2,880 samples at a 30-second interval, which is equivalent to 24 hours of sampled data. The scenarios are then grouped together to create the scenario sets that will be used for the system-level testing. This measurement-based approach forms one of the key strengths of the thesis in that it ensures that the system analysis remains true to the wearable setting.

The scenarios covered in the dataset include heavy-sleep, indoor work, outdoor exercise, mixed commute, cloudy outdoor, indoor exercise, outdoor rest, and mixed scenarios. As mentioned earlier, the choice of these scenarios is not random but reflects the activity and exposure patterns that would be ideal for energy harvesting in a wearable environment due to their reflection of the main activities undertaken by people on a daily basis.

1. Scenario Summary

Scenario	Hours	Mean total (mW)	PV (mW)	TEG (mW)	KEH (mW)	Max KEH (mW)
outdoor_exercise_day	24.0	139.5	138.2	0.816	0.424	3.81
outdoor_rest_day	24.0	125.7	124.9	0.824	0.034	0.63
cloudy_outdoor_day	24.0	70.3	69.3	0.830	0.181	3.44
mixed_environment_day	24.0	41.7	40.7	0.838	0.119	3.15
mixed_commute_day	24.0	23.9	23.0	0.841	0.102	2.13
indoor_work_day	24.0	23.6	22.8	0.838	0.012	0.20
indoor_exercise_day	24.0	18.1	17.0	0.839	0.262	3.40
sleep_heavy	24.0	1.1	0.3	0.847	0.000	0.00

Table 1: Average harvested power across the measured scenario days.

The table describes the main pattern that will continue through all results. Outdoor activities have the highest mean power harvested, owing to the fact that there is more solar energy involved. Motion produces additional energy, though in infrequent peak quantities. Sleep periods show the lowest mean power, because both the solar and kinetic power sources are relatively low, and thus the thermoelectric power forms the base power source in such conditions.

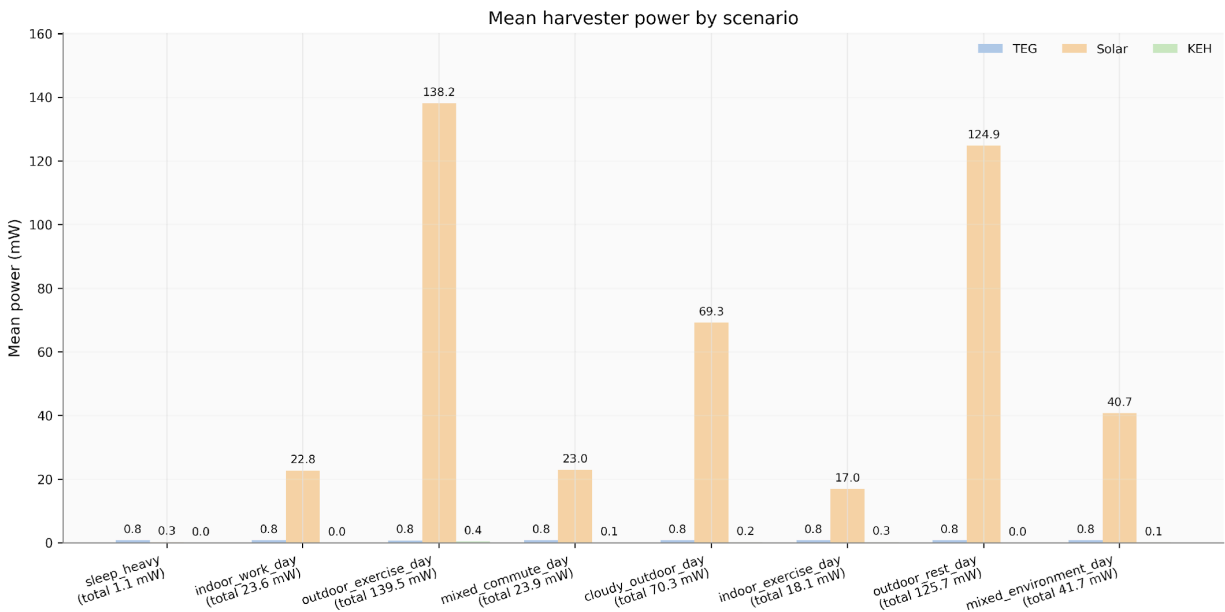


Figure 8: Mean harvested power by scenario and harvester.

This graph depicts the simplest summary of this data set and delivers three important notes. Firstly, solar power outputs the best in case of high energy scenarios. Secondly, the thermoelectric generator provides almost constant values without any significant fluctuations regardless of day type. Lastly, KEH stays low overall, but this figure hides some spikes during active periods which matter the most.

2. How the scenario traces are built

The combined traces are created by adding together the recorded day traces in the scenario catalog. This ensures that the combined traces will be beneficial for the system evaluation, since they allow preserving their innate daily nature and enable the scheduler to work on a larger timescale. For example, the 24 hour traces of indoor exercise doesn't insinuate 24 hours of continuous exercise and instead, 6 hours of continuous exercise in 4 separate instances combined together to give longer traces. It is important to make sure that a clear differentiation exists between the raw day traces and the combined scenario traces, because the former trace the observed behavior of the harvesters, while the latter show the behavior of the wearable device over a whole day.

It is crucial that the difference is made in order not to confuse the measured data with the scheduler data. The measured traces form the foundation of the research, while the scheduler traces are just an additional layer on top of it.

The scheduler model uses a combination of a buffer consisting of a supercapacitor and a battery to store energy. The wearable scheduler decides what actions it needs to take depending on the buffer status, harvesting history, and activity situation. The energy cost model consists of waking up costs, sampling costs, processing costs, transmit base costs, and transmit sample costs.

One of the main conclusions from the model of the scheduler is not only which particular policy wins, but also how well a particular policy matches the energy consumption characteristics of the trace. When the system operates in an energy rich mode, there are several policies that perform almost equally well because the buffer hardly becomes a

bottleneck. On the contrary, when the system operates in an energy constrained manner, the choice of the policy becomes quite important.

Hardware Prototype

The prototype proposed in this paper is composed of three harvesters, each of which operates based on a particular physical property, and provides energy for an overall energy management stack. The first harvester utilizes light incident on the user's body, the second one uses the temperature difference between the user's skin and air, and the third harvester harvests the impact user's heel impact inside the shoes. Therefore, the hardware selections should not only be regarded as separate components but also analyzed in relation to the reasons why the dataset has some specific features, and why power fluctuations are observed in the activity-dependent manner.

1. Photovoltaic Harvester

The photovoltaic path uses the Adafruit 5 V/0.6 W mini solar panel class. The panel in question is a module encapsulated in ETFE that has a power of 0.6 W, open-circuit voltage of 7.09 V, voltage of 6.07 V and a maximum power current of 0.11 A. The effective area of the panel in the model is 42.9025 cm², and in the wearable model, it is placed in a non-rigid clip on the shoulder. It should be noted that in this case, the panel is highly dependent on the rotation of the shoulder, body shadowing, and sunlight direction. In order to ensure that the photovoltaic path in question acts as a light-dependent power source, irradiation, incident angle, tilt of the panel, and its temperature were all considered in this model. Such an inclusion helps understand why photovoltaic traces

prevail in the outdoors and degrade quickly in the indoors and in shade. As can be seen from the thesis figure illustrating photovoltaic responses as a function of the incidence angle, the panel is highly useful but very orientation-dependent.

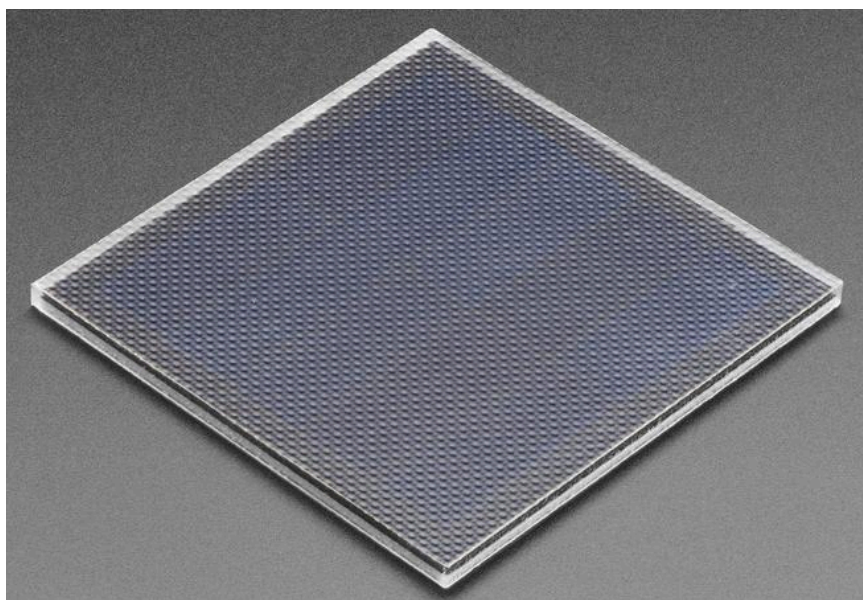


Image 1: Adafruit 5V 0.6W Solar Panel

2. Thermoelectric Harvester

The thermoelectric path relies on the PRMT07-18305-42 module. Hardware testing validates the thermal resistance of 7.0 K/W for the thermoelectric generator, the heatsink (N30-25B recommended in the datasheet) thermal resistance of 9.5 K/W that matches the specified thermal interface, the nominal voltage generation value of 4.2 V, and the cold-start delta-T value of 0.5C. In combination, all of these values make the TEG module the most stable part of the energy budget, though not necessarily the one delivering the most amount of energy. The purpose of the TEG is to generate at least some baseline energy whenever the body temperature rises above that of the environment, including low light and minimal motion situations.

As for the heatsink, its presence in this hardware chain is more than pure aesthetic. It ensures that there is no quick rise in the temperature of the cold side and maintains the thermal gradient necessary for the Seebeck effect. The heatsink is thus considered an integral part of the system being analyzed and not just an extra addition to it. As a result, the output of the thermoelectric energy harvest is modest, though relatively steady, as is consistent with the justification of the TEG provided by the thesis.



Image 2 and 3: N30-25B Heatsink for TEG (left) and PRMT07-18305-42 TEG Module (right)

3. Kinetic Energy Harvester

In particular, the kinetic power source relies on the use of the S129-H5FR-1803YB piezoelectric harvester. As per the product datasheet, the harvester is characterized by the resonance frequency of 175 Hz, capacitance of 22 nF, blocked force of 0.04 N, spring constant of 0.315 N/mm, free deflection of 0.14 mm, device mass of 1.4 g, and the overall size of 70.9 x 10.3378 mm. Since the actual human step rate is significantly lower than the harvester resonance frequency, the shoe movement model must be considered to have the broadband vibration instead of simple shaker-based input.

Tip mass is the most important tuning factor here. This can be seen from the graph provided in the artifacts archive, showing that an increase in tip mass causes the resonant

frequency to fall. For the particular choice of activity traces used in training, the optimal tip mass value appears to be about 19 g. Therefore, the kinetic harvester can be regarded as a resonant device, the performance of which depends on the type of the activity trace and the particular tip mass value used.



Image 4: S129-H5FR-1803YB Module

4. Power Management IC

The power-management IC used for prototyping is the bq25570 energy harvesting module. As per the data sheet and hardware validation, the device acts as a boost charger and a buck converter with battery management for high impedance energy sources, such as photovoltaic cells, thermoelectric generators, and piezoelectric generators through the AC/DC rectifier. In this study, the main role of the device is to illustrate how each output from the harvesters is converted to usable power, as opposed to being raw and uncomparable voltages.

The bq25570 allows for cold-start at microwatts of input power, uses maximum power point tracking and also samples the open-circuit input voltage once every approximately

16 seconds. The device begins energy harvesting once the storage node reaches the operating threshold, and it is also able to provide 100 mA current from VOUT provided sufficient power is present. These properties make the device suitable for use in wearable energy devices since it is capable of handling low voltage from thermoelectric generators while also controlling a supercapacitor and load.

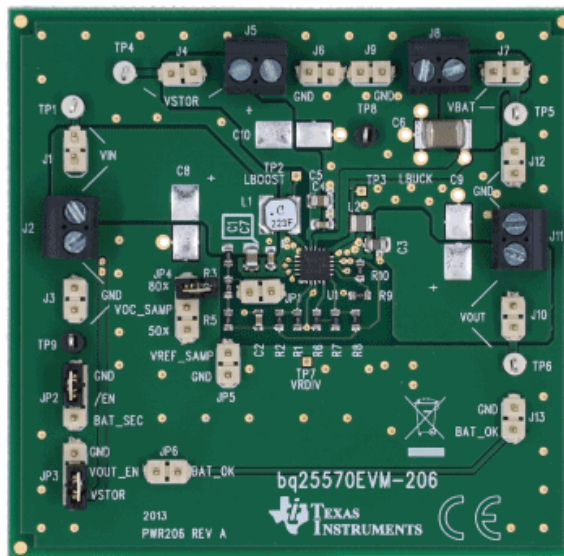


Image 5: BQ25570EVM-206 Development Module

5. Microcontroller and Data Logging Platform

The chosen platform for controlling the system is MSP430FR5994 LaunchPad.

According to the TI user guide provided alongside the platform, the MSP430FR5994 LaunchPad is a development platform for the MSP430FR5994 microcontroller that contains debugging features and energy-measurement capabilities. The MSP430FR5994 microcontroller features 256 KB FRAM, 8 KB of SRAM, operates at a maximum CPU speed of 16 MHz, comes with a low-energy accelerator, and other peripherals necessary for low-power embedded sensing. In addition, the platform comes with a 0.22 F supercapacitor, thus allowing standalone operation without external energy supplies.

As mentioned above, in the current research, the key issue is not only the memory size but the MCU's ability to function intermittently. The FRAM memory allows non-volatile fast write performance, making it a suitable candidate for a lightweight or no-battery sensing node subjected to sporadic energy losses. Besides, the eZ-FET debugger and EnergyTrace++ provided with the board make it convenient for testing prototypes with low energy consumption characteristics. Therefore, the microcontroller and the PMIC form the control core that unites harvesting components, energy storage, and scheduling algorithms within a wearable energy-harvesting system.

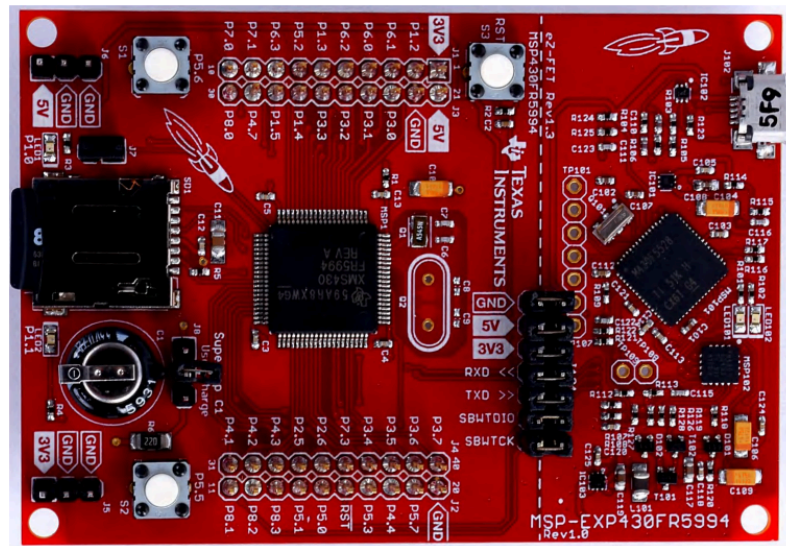


Image 6: MSP430FR5994 LaunchPad Development Kit

Scenario Construction and Interpretation

The scenario files are real empirical traces taken at a frequency of 30 seconds per record. The rows in these scenario files contain the information about the environment together with model results obtained for each of three harvesters. The set of scenarios contains the initial daily files keeping the meaning of every day type: sleeping days, indoors work, outdoors exercise, commute, cloudy outdoors, indoors exercise, outdoors rest and mixed days. This separation is important as the scheduler should be able to handle changes in the ambient environment and not simply an artificial average of all possible days.

Then, the aggregated data goes beyond the limits of daily measurements by connecting several scenarios in a series one after another. It is the right abstraction for the current thesis as the scheduler needs to examine the behavior of battery-like memory through different day types.

Thus, the aggregated data retains its pattern of original measuring files creating a long period of time which tests buffer policy, transmission and sampling policies.

The combined scenario families in the artifact are organized as follows.

Scenario family	Constituent day traces	Purpose
active_commute	mixed_commute_day + outdoor_exercise_day	Stresses motion-heavy transitions.
solar_only	outdoor_rest_day + cloudy_outdoor_day	Isolates daylight dependence.
rest_only	sleep_heavy + indoor_work_day + outdoor_rest_day	Tests low-motion, mixed-light behavior.
keh_plus_teg	indoor_exercise_day + mixed_commute_day + outdoor_exercise_day	Highlights motion and body-heat pairing.
no_solar	sleep_heavy + indoor_work_day + indoor_exercise_day + mixed_environment_day	Exercises the system with little or no sunlight.
all_sources	All eight day traces	Gives the broadest combined operating window.

Table 3: Scenario comparison based on traces

This organization has the benefit of framing the thesis as a comparison of operating regimes. The solar-only group functions as a daytime experiment, the no-solar group acts as a scarcity experiment, while the all-sources group serves as a full stress test. Together, they make it possible for the results section to consider not only the sum of power generated by the individual harvesters but also the scheduler under changing profiles throughout the day.

The summed traces do not imply continuous activity, continuous rest, or continuous commuting. Instead, they represent day-level stitched datasets that maintain the innate daily nature of the data collection. This allows for the realistic assumptions made within the thesis about wearables without transforming the dataset into a mere signal generator.

Results

There are two major layers within the results chapter. First of all, there is harvester characterization which discusses how individual sources operate independently, as well as changes in behavior of the source combination based on scenario. Secondly, there is the analysis of scheduler policy behavior when combined harvesting is introduced to the storage model. While both layers shed insight into hybrid harvesting, the first one is more significant since it is built on measurements and realistic model parameters.

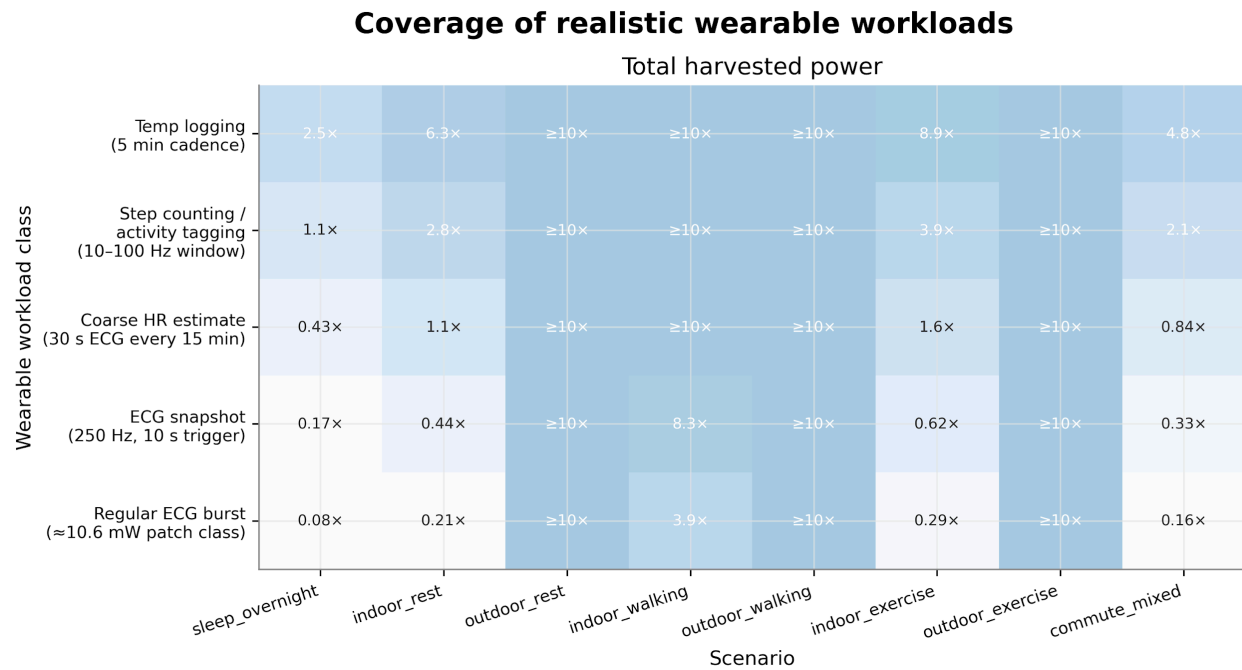


Figure 9: Overall wearable hybrid energy-harvesting system overview.

1. PV Plots

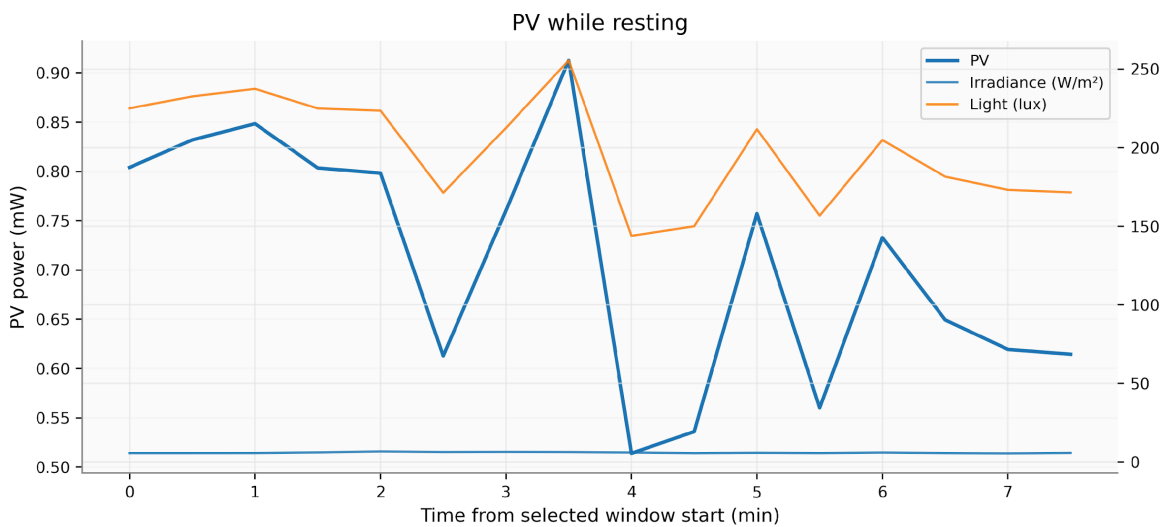


Figure 10: Photovoltaic power in a rest-oriented scenario.

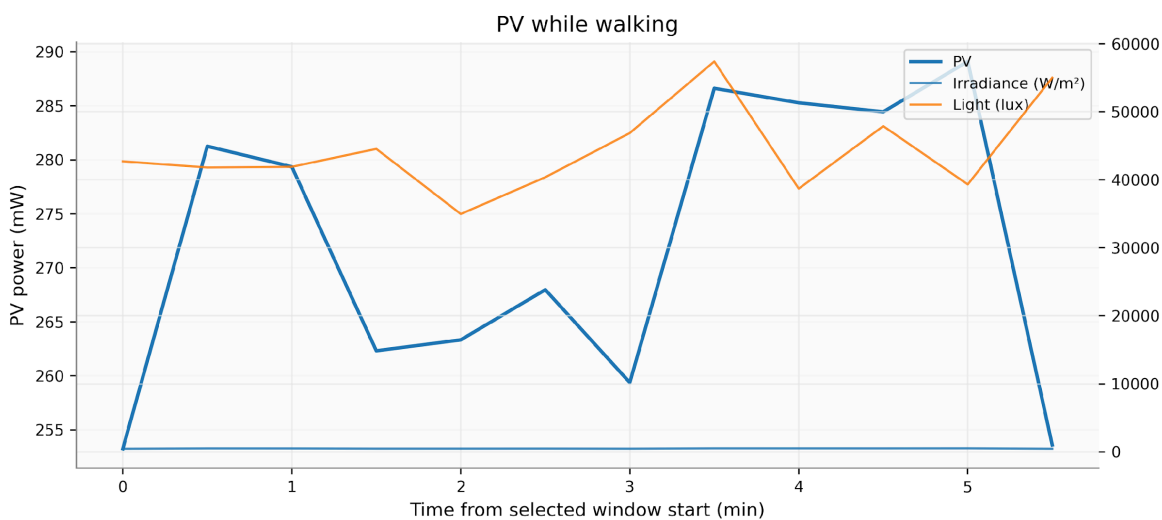


Figure 11: Photovoltaic power in a walking scenario.

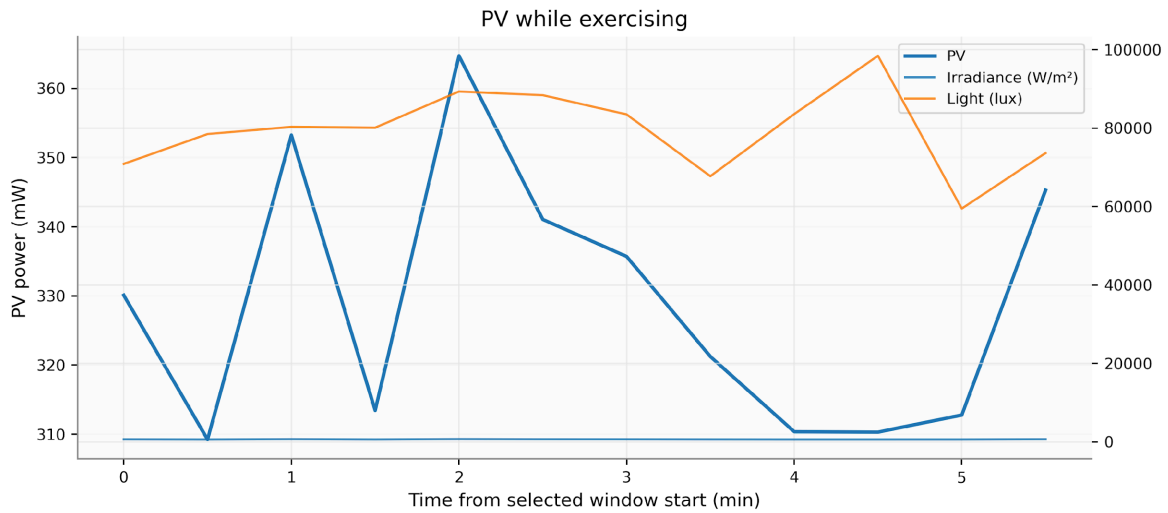


Figure 12: Photovoltaic power in an exercising scenario.

For PV plots, the presented behavior is expected as solar power should be plentiful in illuminated areas and absent in indoor environments or under shade. Therefore, PV is used as the main source of power when exercising, resting, or working outdoors in a cloudy environment; otherwise, it is less popular. In other words, PV is very useful but not always dependable

2. TEG Plots



Figure 13: Thermoelectric power in a rest-oriented scenario.

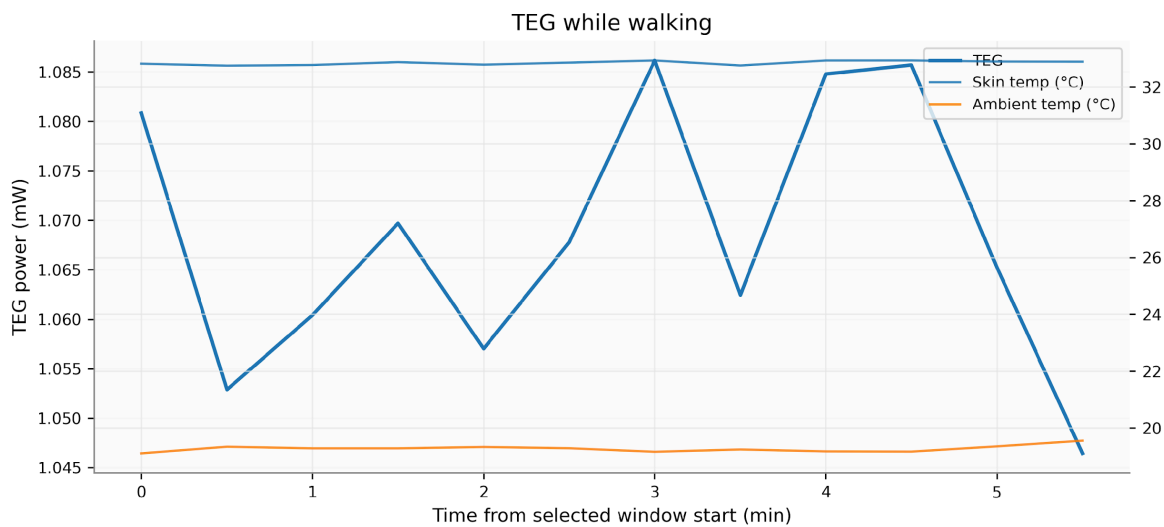


Figure 14: Thermoelectric power in a walking scenario.

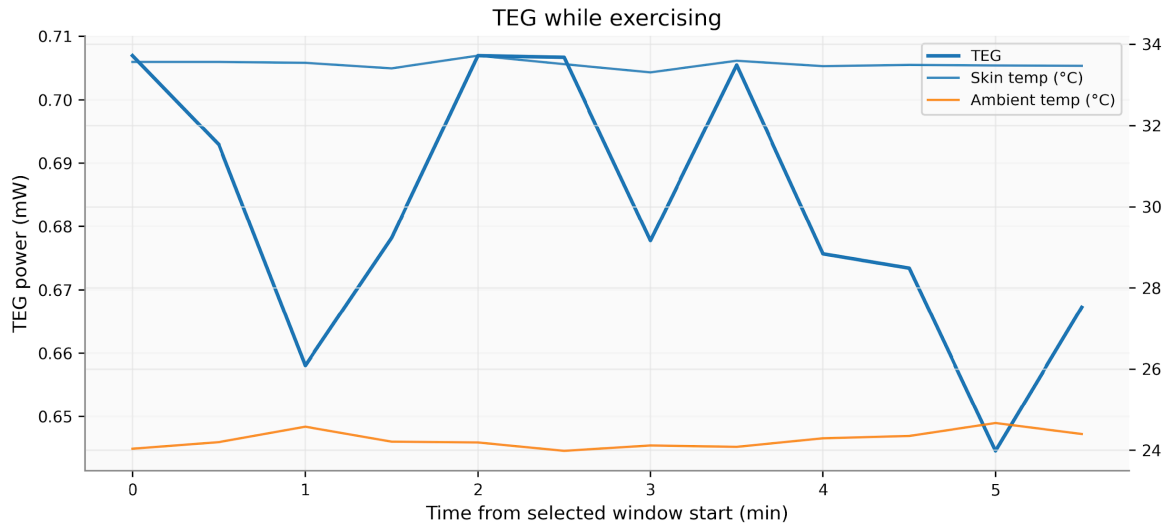


Figure 15: Thermoelectric power in an exercising scenario.

The TEG plots are clearly showing that TEG does not generate as much power as solar but this should not be considered a flaw since the reason behind using the harvester in the first place is its stability. The thermoelectric source ensures a steady baseline for powering the wearable device when both motion and light are lacking. The primary role of TEG is as the element that stops the energy level from falling to zero, and not as the main provider of power constantly.

3. KEH Plots

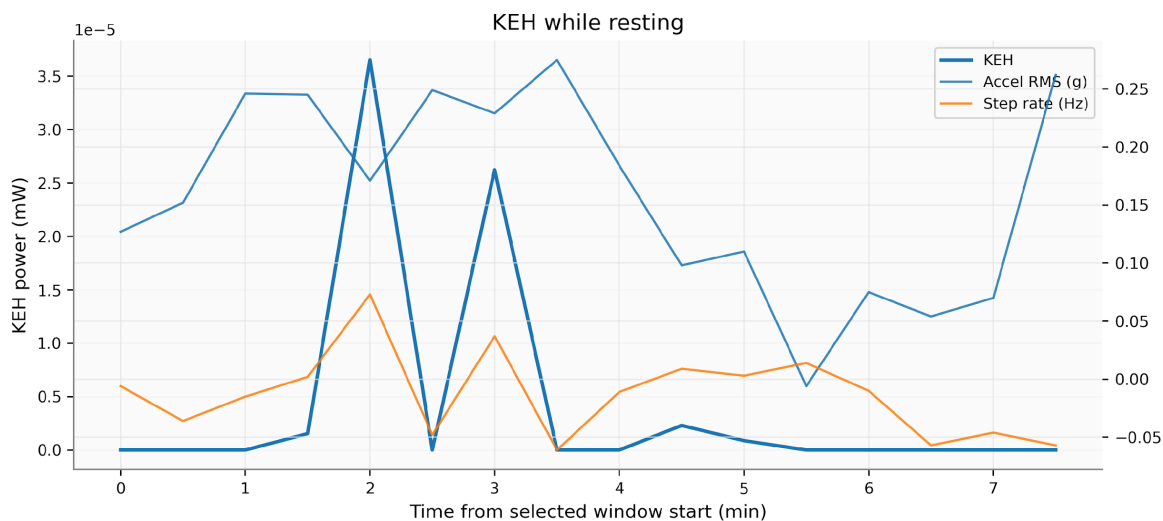


Figure 16: Kinetic power in a rest-oriented scenario.

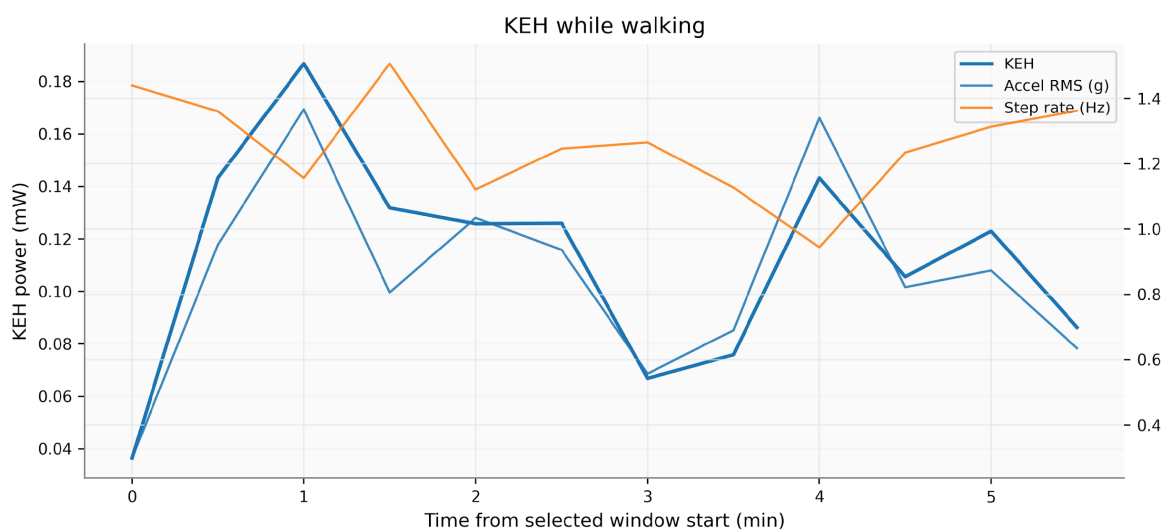


Figure 17: Kinetic power in a walking scenario.

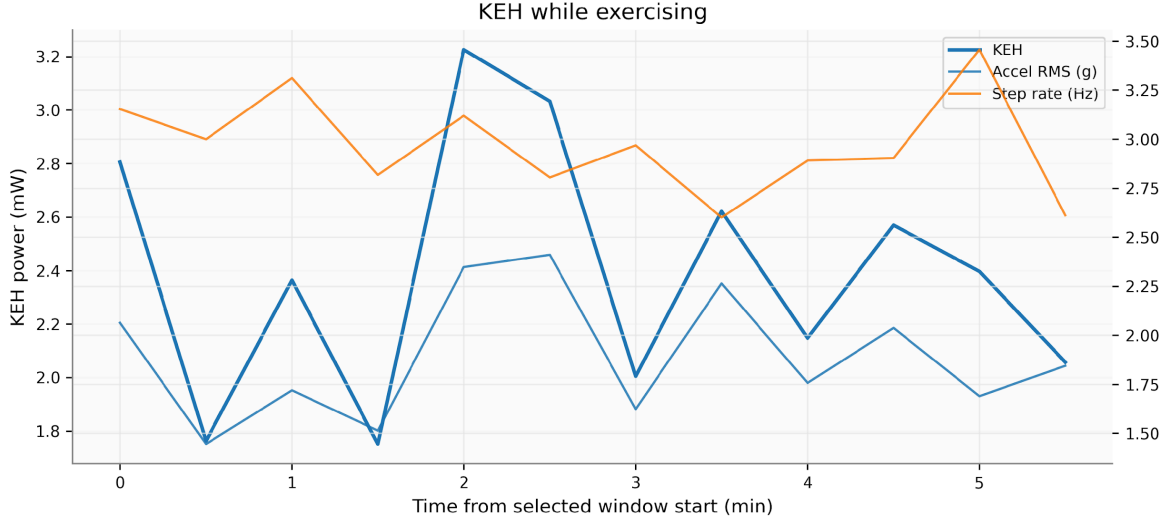


Figure 18: Kinetic power in an exercising scenario.

The KEH graphs display the sporadic nature of kinetic energy harvesting. Even though the average value may appear lower than TEG in most situations, the highest output occurs during times when the user is moving. Hence, KEH is ideal for sensing or transmitting information quickly. It also explains the importance of choosing the right proof mass since the device will respond to the available motion frequencies. Without the right tip mass, even the peaks of this harvester would be lower than the TEG.

4. Policy Comparison on the Combined Trace

Policy	Samples	Tx	Mean SC V	Final SC V	Uptime fraction
periodic	11520	1497	3.298	3.300	0.500
threshold	23005	2430	3.298	3.300	0.998
predictive	11520	1497	3.298	3.300	0.500
opportunistic	17133	1951	3.298	3.300	0.744
energy_neutral	23009	2436	3.298	3.300	0.999

Table 2: Policy comparison with the traces

Periodic and predictive policies only sample in half of all trace steps, whereas threshold and energy-neutral policies nearly always sample. Opportunistic sampling falls in between these extremes. Thus, what matters is not whether the system samples at some

point in time, but how aggressive the policies are when there is sufficient energy to support sampling.

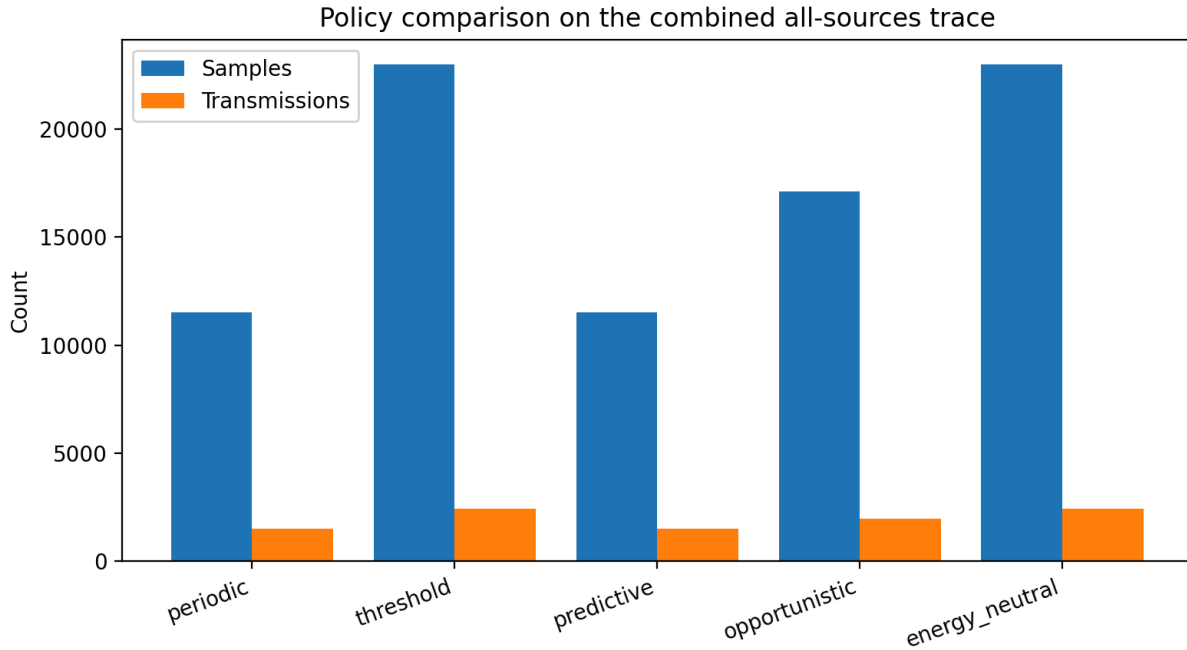


Figure 19: Corrected policy comparison on the combined all-sources trace.

This figure puts the whole thesis into perspective. Threshold and energy-neutral policies are the most liberal, opportunistic policies fall in between, and periodic/predictive policies are the most conservative. Of course, it would be incorrect for this thesis to draw attention away from the similarity between the different policy families. On the contrary, the correct conclusion is that the different policy classes exploit the margin differently.

5. Discussion of Scheduler Layer

These scheduling outcomes need to be understood at a system level interpretations of the measurements taken. In cases where there is high power that has been harvested, the buffer rarely becomes a problem, thereby causing convergence amongst several policies. As such, when power is low, policies begin to diverge in results. This is the logic behind

the inclusion of the scheduler layer in the thesis, although this layer does not constitute the core part of the contributions made to the field of study.

From the scheduler outcomes, the implication is clear that while a hybrid system is capable enough to withstand a significant sensing duty cycle by the wearable within good conditions, energy awareness in the control policy is important. Therefore, the hybrid model cannot be considered in terms of energy maximization alone, but as protection from the times that the energy source is low.

Conclusion and Future Work

This thesis establishes that there are different roles played by the three harvesters in a wearable hybrid energy system. Photovoltaic harvester plays the role of the main energy source during light rich environments outdoors; the thermoelectric harvester plays the role of providing energy on a small scale, consistently, while the kinetic energy harvester delivers short bursts that depend on motion dynamics and proof mass configuration. In summary, it should be clear that there is no question of one harvester replacing the other in the wearable system, but all three sources play different roles in the best performance of the diurnal period.

This is confirmed by the scheduler result presented in the artifact in the energy rich case since many policies result into the same result since the energy buffer does not become the bottleneck. On the contrary, in the energy poor case, it is evident that the distinction between the policies becomes significant as the energy buffer cannot withstand poor energy harvest without affecting the sampling and transmission process. This forms the main idea of the thesis, which states that

the behavior of a wearable depends on the alignment between task schedule and energy harvesting.

However, there are clear limitations of the study. The current thesis is based on estimated energy costs, certain hardware stacks and limited day traces. Therefore, the results can only be considered reliable within the framework of hardware-based simulations rather than universally applicable design guidelines for wearables. In fact, while harvesting schemes are quite realistic and well substantiated with data, the system requires certain choices of components and storage capacities along with threshold scheduler values.

Further research must follow three lines of thought. Firstly, harvesting modeling must be developed further through longer validation with varied weather conditions, different postures and gaits of a wearer. It is also important to note that the scenario testing was exclusively done by the author of this paper and not by any other participants. Secondly, scheduler must be improved in terms of incorporating predictive logic, which allows it to make decisions in accordance with not only the state of the buffer but also its unpredictability. Lastly, a prototype must be perfected to resemble an actual wearable node, including PMIC, storage device, MCU and sensing task within one experimental setup.

A further extension could include comparing different ways to place and package the components. For instance, the PV panel might be evaluated on several body locations and angles; the TEG might be placed with other heatsinks or enclosures; the kinetic harvester might be tested with new proof masses or placements on shoes. This method will allow separation between the body placement effect and the effect of the transducer itself. Additionally there's no off the shelf wearable kinetic harvester in the market and even the component I used is meant for more industrial use cases as evident by its resonant frequencies.

The general lesson from this research is that hybrid wearable energy harvesting is most credible when the hardware, scenario building, and scheduling algorithm are all considered together. This combination is the importance of this thesis, where we use real-world hardware, real-world traces, and see if this hybrid system can still do its job of useful sensing at different ambient energy levels. It can, but only if we consider the system as a whole and not as three separate harvesters.

Appendix: Code

core_harvesters.py

```
#!/usr/bin/env python3
"""Core models for the hybrid wearable energy-harvesting simulator.
"""
from __future__ import annotations

from dataclasses import dataclass
from pathlib import Path
from typing import Dict, List, Tuple
import hashlib
import json
import math

import numpy as np
import pandas as pd

# -----
# Hardware validated constants
# -----
# TEG: PRMT07-18305-42
TEG_R_THERMAL_OHMS = 7.0
HEATSINK_R_THERMAL_OHMS = 9.5 # N30-25B recommendation from the datasheet
note
TEG_VOUT_V = 4.2
TEG_COLD_START_DELTA_T_C = 0.5

# PV: Adafruit 5 V / 0.6 W mini solar panel class.
PV_P_STC_W = 0.60
PV_VMP_V = 6.07
PV_IMP_A = 0.11
PV_VOC_V = 7.09
PV_AREA_CM2 = 65.5 * 65.5 / 100.0
PV_TEMP_COEFF_PER_C = -0.004
PV_INDOOR_LUX_TO_WM2 = 0.0079

# Piezo: S129-H5FR-1803YB values.
```

```

PIEZO_RESONANT_FREQ_HZ = 175.0
PIEZO_CAP_NF = 22.0
PIEZO_BLOCKED_FORCE_N = 0.04
PIEZO_SPRING_CONST_N_PER_MM = 0.315
PIEZO_RATED_DRIVE_V = 200.0
PIEZO_MAX_DRIVE_V_AT_RESONANCE = 10.0
PIEZO_FREE_DEFLECTION_MM = 0.14
PIEZO_MASS_G = 1.4
PIEZO_LENGTH_MM = 70.9
PIEZO_WIDTH_MM = 10.3378

```

@dataclass

```

class TEGConfig:
    r_teg: float = TEG_R_THERMAL_OHMS
    r_hs: float = HEATSINK_R_THERMAL_OHMS
    vout_v: float = TEG_VOUT_V
    cold_start_delta_t_c: float = TEG_COLD_START_DELTA_T_C
    eta_pm: float = 0.86

```

@dataclass

```

class PVConfig:
    p_stc_w: float = PV_P_STC_W
    v_mp_v: float = PV_VMP_V
    i_mp_a: float = PV_IMP_A
    voc_v: float = PV_VOC_V
    temp_coeff_per_c: float = PV_TEMP_COEFF_PER_C
    indoor_lux_to_wm2: float = PV_INDOOR_LUX_TO_WM2
    eta_pm: float = 0.92

```

@dataclass

```

class PiezoConfig:
    f_res_hz: float = PIEZO_RESONANT_FREQ_HZ
    c_p_nf: float = PIEZO_CAP_NF
    spring_const_n_per_mm: float = PIEZO_SPRING_CONST_N_PER_MM
    mass_g: float = PIEZO_MASS_G
    length_mm: float = PIEZO_LENGTH_MM
    width_mm: float = PIEZO_WIDTH_MM

```

```

max_tip_mass_g: float = 500.0
eta_rectifier: float = 0.82
eta_pm: float = 0.88

```

@dataclass

```

class StorageConfig:
    sc_cap_f: float = 0.50
    sc_v_min: float = 2.2
    sc_v_max: float = 3.3
    battery_v_nom: float = 3.7
    battery_v_min: float = 3.0
    battery_v_max: float = 4.2
    battery_capacity_mah: float = 30.0
    sc_charge_to_battery_v: float = 3.05
    eta_sc_charge: float = 0.92
    eta_bat_charge: float = 0.90
    eta_discharge: float = 0.93
    leakage_sc_w: float = 3.0e-6
    leakage_bat_w: float = 0.8e-6

```

@dataclass

```

class SchedulerConfig:
    sample_energy_j: float = 0.18e-3
    process_energy_j: float = 0.08e-3
    tx_base_energy_j: float = 1.8e-3
    tx_per_sample_energy_j: float = 0.12e-3
    wake_energy_j: float = 0.05e-3
    idle_energy_w: float = 12e-6
    emergency_event_energy_j: float = 0.06e-3
    base_sample_period_s: int = 60
    high_energy_period_s: int = 20
    low_energy_period_s: int = 300
    batch_size: int = 12
    sc_high_threshold_v: float = 2.95
    sc_low_threshold_v: float = 2.45
    bat_low_threshold_v: float = 3.25

```



```

# -----
# Helpers
# -----

def clamp(x: float, lo: float, hi: float) -> float:
    return max(lo, min(hi, x))

def energy_in_capacitor(c_f: float, v: float) -> float:
    return 0.5 * c_f * v * v

def voltage_from_energy(c_f: float, e_j: float) -> float:
    if e_j <= 0:
        return 0.0
    return math.sqrt(2.0 * e_j / c_f)

def _interp(x: np.ndarray, xp: np.ndarray, fp: np.ndarray) -> np.ndarray:
    return np.interp(x, xp, fp, left=fp[0], right=fp[-1])

# -----
# Harvester models
# -----

class TEGModel:
    """Empirical wrapper around the PRMT07-18305-42 curve.
    """

    def __init__(self, cfg: TEGConfig | None = None):
        self.cfg = cfg or TEGConfig()
        self._delta_t_pts = np.array([0.0, 0.5, 1, 2, 5, 10, 15, 20, 25,
30, 35], dtype=float)
        self._current_ma_pts = np.array([0.0, 0.0, 0.02, 0.08, 0.25, 0.55,
0.90, 1.20, 1.50, 1.80, 2.00], dtype=float)

    def delta_t_teg_c(self, skin_c: np.ndarray, ambient_c: np.ndarray) ->
np.ndarray:
        source_delta = np.maximum(0.0, skin_c - ambient_c)
        return source_delta * (self.cfg.r_teg / (self.cfg.r_teg +
self.cfg.r_hs))

```

```

    def open_circuit_voltage(self, delta_t_teg_c: np.ndarray) ->
np.ndarray:
    return 0.018 * np.maximum(0.0, delta_t_teg_c)

    def output_power_w(self, skin_c: np.ndarray, ambient_c: np.ndarray) ->
np.ndarray:
    dt = self.delta_t_teg_c(skin_c, ambient_c)
    current_ma = _interp(dt, self._delta_t_pts, self._current_ma_pts)
    power_w = (current_ma / 1000.0) * self.cfg.vout_v
    power_w = np.where(dt < self.cfg.cold_start_delta_t_c, 0.0,
power_w * self.cfg.eta_pm)
    return power_w

    def summary_curve(self) -> pd.DataFrame:
    dt = self._delta_t_pts.copy()
    i_ma = self._current_ma_pts.copy()
    return pd.DataFrame({"delta_t_c": dt, "output_current_ma": i_ma,
"output_power_mw": i_ma * self.cfg.vout_v})

class PVModel:
    """Simple shoulder-mounted solar panel model.

    The model uses a clear-sky irradiance term when available, a cosine
    incidence
    factor for shoulder orientation, an indoor-lux fallback, and a mild
    thermal
    derate.
    """

    def __init__(self, cfg: PVConfig | None = None):
        self.cfg = cfg or PVConfig()

    def irradiance_to_power_w(
        self,
        irradiance_wm2: np.ndarray,
        incidence_deg: np.ndarray,
        cell_temp_c: np.ndarray,
        indoor_lux: np.ndarray | None = None,

```

```

) -> np.ndarray:
    inc = np.deg2rad(np.clip(incidence_deg, 0.0, 90.0))
    geo = np.maximum(0.0, np.cos(inc))
    if indoor_lux is None:
        g = np.maximum(0.0, irradiance_wm2)
    else:
        indoor_wm2 = np.maximum(0.0, indoor_lux) *
self.cfg.indoor_lux_to_wm2
        g = np.where(irradiance_wm2 > 20.0, irradiance_wm2,
indoor_wm2)
        temp_factor = 1.0 + self.cfg.temp_coeff_per_c * np.maximum(-20.0,
cell_temp_c - 25.0)
        temp_factor = np.clip(temp_factor, 0.75, 1.05)
        p = self.cfg.p_stc_w * (g / 1000.0) * geo * temp_factor
        return np.clip(p * self.cfg.eta_pm, 0.0, self.cfg.p_stc_w * 1.08)

```

```

class PiezoModel:

```

```

    """Piezoelectric shoe-harvester model.

```

```

    The S129 transducer is resonant at roughly 175 Hz, which is far above
a raw

```

```

    step cadence. For the wearable shoe use case we therefore model the
shoe as a

```

```

    broadband impact exciter: cadence controls how often impacts occur,
while the

```

```

    dominant vibration band is estimated from the activity and
acceleration.

```

```

    Tip mass is treated as a tunable proof mass. The model intentionally
keeps the

```

```

    resonance sweep explicit so that the trade-off between lower resonant
frequency

```

```

    and added mass can be quantified and reported.

```

```

    """

```

```

    def __init__(self, cfg: PiezoConfig | None = None):

```

```

        self.cfg = cfg or PiezoConfig()

```

```

    @property

```

```

def beam_effective_mass_g(self) -> float:
    k_n_per_m = self.cfg.spring_const_n_per_mm * 1000.0
    m_eff_kg = k_n_per_m / ((2.0 * math.pi * self.cfg.f_res_hz) ** 2)
    return m_eff_kg * 1000.0

def resonant_frequency_with_tip_mass(self, tip_mass_g: float |
np.ndarray) -> np.ndarray:
    m_eff = self.beam_effective_mass_g
    tip_mass = np.asarray(tip_mass_g, dtype=float)
    total = m_eff + np.maximum(0.0, tip_mass)
    return self.cfg.f_res_hz * np.sqrt(m_eff / total)

def required_tip_mass_for_target_freq(self, target_freq_hz: float) ->
float:
    m_eff = self.beam_effective_mass_g
    if target_freq_hz <= 0:
        return float("inf")
    total = m_eff * (self.cfg.f_res_hz / target_freq_hz) ** 2
    return max(0.0, total - m_eff)

def _dominant_excitation_hz(self, activity: np.ndarray, cadence_hz:
np.ndarray, accel_rms_g: np.ndarray) -> np.ndarray:
    activity = np.asarray(activity)
    cadence_hz = np.asarray(cadence_hz, dtype=float)
    accel_rms_g = np.asarray(accel_rms_g, dtype=float)
    f = np.zeros_like(cadence_hz, dtype=float)

    moving = ~np.isin(activity, ["sleep"])
    f[moving] = 4.5 + 3.8 * cadence_hz[moving] + 1.7 *
np.clip(accel_rms_g[moving], 0.0, 4.0)
    f = np.where(np.isin(activity, ["indoor_rest", "outdoor_rest"]),
3.0 + 0.4 * cadence_hz, f)
    f = np.where(activity == "commute", 7.5 + 2.9 * cadence_hz + 1.2 *
np.clip(accel_rms_g, 0.0, 4.0), f)
    f = np.where(np.isin(activity, ["indoor_walk", "outdoor_walk"]),
6.0 + 3.2 * cadence_hz + 1.0 * np.clip(accel_rms_g, 0.0, 4.0), f)
    f = np.where(np.isin(activity, ["indoor_exercise",
"outdoor_exercise"]), 8.0 + 3.6 * cadence_hz + 1.4 * np.clip(accel_rms_g,
0.0, 4.0), f)
    return np.where(activity == "sleep", 0.0, np.clip(f, 0.0, 80.0))

```

```

def _activity_scale(self, activity: np.ndarray) -> np.ndarray:
    activity = np.asarray(activity)
    scale = np.ones_like(activity, dtype=float)
    scale = np.where(activity == "sleep", 0.0, scale)
    scale = np.where(activity == "indoor_rest", 0.06, scale)
    scale = np.where(activity == "outdoor_rest", 0.08, scale)
    scale = np.where(activity == "commute", 0.65, scale)
    scale = np.where(activity == "indoor_walk", 0.95, scale)
    scale = np.where(activity == "outdoor_walk", 1.05, scale)
    scale = np.where(activity == "indoor_exercise", 1.55, scale)
    scale = np.where(activity == "outdoor_exercise", 1.75, scale)
    return scale

def power_w(
    self,
    activity: np.ndarray,
    cadence_hz: np.ndarray,
    accel_rms_g: np.ndarray,
    dt_s: float = 1.0,
    tip_mass_g: float | np.ndarray = 0.0,
) -> np.ndarray:
    activity = np.asarray(activity)
    cadence_hz = np.asarray(cadence_hz, dtype=float)
    accel_rms_g = np.asarray(accel_rms_g, dtype=float)
    tip_mass = np.asarray(tip_mass_g, dtype=float)
    tip_mass = np.broadcast_to(tip_mass, cadence_hz.shape)

    f_dom = self._dominant_excitation_hz(activity, cadence_hz,
    accel_rms_g)
    f_res = self.resonant_frequency_with_tip_mass(tip_mass)

    # The excitation band is broad because shoe impacts are not
    perfectly sinusoidal.
    sigma_hz = 11.5
    resonance_gain = np.exp(-0.5 * ((f_dom - f_res) / sigma_hz) ** 2)

    activity_scale = self._activity_scale(activity)
    accel_factor = np.power(np.clip(accel_rms_g, 0.0, 5.0) / 1.2,
1.20)

```

```

        cadence_factor = np.clip(cadence_hz / 2.0, 0.0, 2.2)

        # A very small tip mass barely changes the proof-mass dynamics; a
        very large
        # mass shifts the resonance lower, but also becomes less efficient
        once it
        # moves too far below the dominant impact band.
        mass_gain = 1.0 + 0.018 * np.sqrt(np.clip(tip_mass, 0.0,
self.cfg.max_tip_mass_g)) + 0.00045 * np.clip(tip_mass, 0.0,
self.cfg.max_tip_mass_g)
        comfort_penalty = np.exp(-np.clip(tip_mass, 0.0,
self.cfg.max_tip_mass_g) / 900.0)

        # Base scale chosen so running/outdoor exercise can exceed the TEG
        when active.
        p_w = 0.00070 * activity_scale * accel_factor * cadence_factor *
        resonance_gain * mass_gain * comfort_penalty
        p_w *= self.cfg.eta_rectifier * self.cfg.eta_pm
        return np.clip(p_w, 0.0, 0.02) / max(dt_s, 1e-9)

def step_energy_j(
    self,
    activity: np.ndarray,
    cadence_hz: np.ndarray,
    accel_rms_g: np.ndarray,
    tip_mass_g: float | np.ndarray = 0.0,
) -> np.ndarray:
    return self.power_w(activity, cadence_hz, accel_rms_g, dt_s=1.0,
tip_mass_g=tip_mass_g)

def recommended_tip_mass_g(
    self,
    activity: str = "outdoor_exercise",
    cadence_hz: float = 2.85,
    accel_rms_g: float = 1.90,
    sweep_max_g: float | None = None,
    step_g: float = 1.0,
) -> Tuple[float, pd.DataFrame]:
    max_g = self.cfg.max_tip_mass_g if sweep_max_g is None else
min(float(sweep_max_g), self.cfg.max_tip_mass_g)

```

```

masses = np.arange(0.0, max_g + step_g, step_g)
acts = np.full_like(masses, activity, dtype=object)
cad = np.full_like(masses, cadence_hz, dtype=float)
acc = np.full_like(masses, accel_rms_g, dtype=float)
powers = self.power_w(acts, cad, acc, tip_mass_g=masses)
df = pd.DataFrame({
    "tip_mass_g": masses,
    "resonant_frequency_hz":
[self.resonant_frequency_with_tip_mass(m) for m in masses],
    "power_w": powers,
})
best_idx = int(df["power_w"].to_numpy().argmax())
return float(df.iloc[best_idx]["tip_mass_g"]), df

# -----
# Hybrid buffer
# -----

class HybridBuffer:
    """Supercapacitor + battery hybrid storage model."""

    def __init__(self, cfg: StorageConfig | None = None):
        self.cfg = cfg or StorageConfig()
        self.sc_e_j = energy_in_capacitor(self.cfg.sc_cap_f,
self.cfg.sc_v_min)
        self.bat_e_j = self.battery_max_energy_j * 0.35

    @property
    def sc_e_max_j(self) -> float:
        return energy_in_capacitor(self.cfg.sc_cap_f, self.cfg.sc_v_max)

    @property
    def sc_e_min_j(self) -> float:
        return energy_in_capacitor(self.cfg.sc_cap_f, self.cfg.sc_v_min)

    @property
    def battery_max_energy_j(self) -> float:
        return (self.cfg.battery_capacity_mah / 1000.0) *
self.cfg.battery_v_nom * 3600.0

```

```

@property
def battery_min_energy_j(self) -> float:
    frac = (self.cfg.battery_v_min / self.cfg.battery_v_max) ** 2
    return self.battery_max_energy_j * frac

def reset(self, sc_v: float | None = None, bat_frac: float = 0.35) ->
None:
    self.sc_e_j = energy_in_capacitor(self.cfg.sc_cap_f, sc_v if sc_v
is not None else self.cfg.sc_v_min)
    self.bat_e_j = self.battery_max_energy_j * bat_frac

def sc_voltage(self) -> float:
    return voltage_from_energy(self.cfg.sc_cap_f, self.sc_e_j)

def battery_voltage(self) -> float:
    frac = max(0.0, min(1.0, self.bat_e_j /
self.battery_max_energy_j))
    return self.cfg.battery_v_min + math.sqrt(frac) *
(self.cfg.battery_v_max - self.cfg.battery_v_min)

def total_energy_j(self) -> float:
    return self.sc_e_j + self.bat_e_j

def charge(self, harvested_j: float) -> Dict[str, float]:
    harvested_j = max(0.0, harvested_j)
    sc_room = max(0.0, self.sc_e_max_j - self.sc_e_j)
    sc_take = min(sc_room, harvested_j * self.cfg.eta_sc_charge)
    self.sc_e_j += sc_take
    rem = harvested_j - sc_take / max(self.cfg.eta_sc_charge, 1e-9)

    bat_take = 0.0
    if rem > 0.0:
        bat_room = max(0.0, self.battery_max_energy_j - self.bat_e_j)
        bat_take = min(bat_room, rem * self.cfg.eta_bat_charge)
        self.bat_e_j += bat_take
        rem = rem - bat_take / max(self.cfg.eta_bat_charge, 1e-9)

    return {"sc_charged_j": sc_take, "bat_charged_j": bat_take,
"spilled_j": max(0.0, rem)}

```



```

def discharge(self, need_j: float) -> Dict[str, float]:
    need_j = max(0.0, need_j)
    from_sc = min(max(0.0, self.sc_e_j - self.sc_e_min_j), need_j /
max(self.cfg.eta_discharge, 1e-9))
    self.sc_e_j -= from_sc
    remaining_need = need_j - from_sc * self.cfg.eta_discharge

    from_bat = 0.0
    if remaining_need > 0:
        from_bat = min(max(0.0, self.bat_e_j -
self.battery_min_energy_j), remaining_need / max(self.cfg.eta_discharge,
1e-9))
        self.bat_e_j -= from_bat
        remaining_need -= from_bat * self.cfg.eta_discharge

    return {"from_sc_j": from_sc, "from_bat_j": from_bat, "unmet_j":
max(0.0, remaining_need)}

# -----
# Scheduler / task model
# -----

class WearableScheduler:
    """Energy-aware scheduler for a wearable medical/environmental
    monitor."""

    def __init__(self, cfg: SchedulerConfig | None = None):
        self.cfg = cfg or SchedulerConfig()

    def decide_period_s(self, sc_v: float, bat_v: float,
predicted_total_w: float, activity: str) -> int:
        if sc_v >= 2.98 or bat_v >= 4.0 or predicted_total_w > 1.2e-3:
            return self.cfg.high_energy_period_s
        if sc_v <= self.cfg.sc_low_threshold_v or bat_v <=
self.cfg.bat_low_threshold_v:
            return self.cfg.low_energy_period_s
        if activity in {"indoor_rest", "sleep"}:
            return max(self.cfg.base_sample_period_s, 120)
        return self.cfg.base_sample_period_s

```

```

def step(
    self,
    buffer: HybridBuffer,
    t_idx: int,
    row: pd.Series,
    predicted_total_w: float,
    last_sample_t: int,
    pending_samples: int,
    tick_s: float = 1.0,
) -> Dict[str, object]:
    sc_v = buffer.sc_voltage()
    bat_v = buffer.battery_voltage()
    activity = str(row.get("activity", "unknown"))
    period_s = self.decide_period_s(sc_v, bat_v, predicted_total_w,
activity)
    period_ticks = max(1, int(math.ceil(period_s / max(tick_s,
1e-9))))

    do_sample = (t_idx - last_sample_t) >= period_ticks
    do_tx = False
    emergency = False

    if row.get("activity", "") in {"outdoor_walk", "outdoor_exercise",
"commute"} and row.get("accel_rms_g", 0.0) > 1.5:
        emergency = True

    idle_need = self.cfg.idle_energy_w * tick_s
    buffer.discharge(idle_need)

    if do_sample:
        sample_need = self.cfg.wake_energy_j +
self.cfg.sample_energy_j + self.cfg.process_energy_j
        used = buffer.discharge(sample_need)
        if used["unmet_j"] <= 0.0:
            pending_samples += 1
            last_sample_t = t_idx
            do_sample = True
        else:
            do_sample = False

```

```

        if pending_samples >= self.cfg.batch_size:
            tx_need = self.cfg.tx_base_energy_j + pending_samples *
self.cfg.tx_per_sample_energy_j
            used = buffer.discharge(tx_need)
            if used["unmet_j"] <= 0.0:
                do_tx = True
                pending_samples = 0

        if emergency and buffer.sc_voltage() >=
self.cfg.sc_high_threshold_v and buffer.battery_voltage() >=
self.cfg.bat_low_threshold_v:
            used = buffer.discharge(self.cfg.emergency_event_energy_j)
            if used["unmet_j"] <= 0.0:
                do_tx = True

    return {
        "do_sample": do_sample,
        "do_tx": do_tx,
        "pending_samples": pending_samples,
        "last_sample_t": last_sample_t,
        "period_s": period_s,
        "sc_v": buffer.sc_voltage(),
        "bat_v": buffer.battery_voltage(),
    }

```

```

def smooth_noise(rng: np.random.Generator, n: int, scale: float, window:
int = 31) -> np.ndarray:
    white = rng.normal(0.0, scale, n)
    if n < 3 or window <= 1:
        return white
    kernel = np.ones(window) / window
    return np.convolve(white, kernel, mode="same")

```

scheduler.py

```
#!/usr/bin/env python3
"""Build combined harvester scenarios and run the comparison scheduler."""
from __future__ import annotations

from pathlib import Path
import argparse
import json
from typing import Dict, List

import numpy as np
import pandas as pd

from core_harvesters import TEGModel, PVMModel, PiezoModel, HybridBuffer,
WearableScheduler

SCENARIO_MAP = {
    "all_sources": ["sleep_heavy", "indoor_work_day",
"outdoor_exercise_day", "mixed_commute_day", "cloudy_outdoor_day",
"indoor_exercise_day", "outdoor_rest_day", "mixed_environment_day"],
    "no_solar": ["sleep_heavy", "indoor_work_day", "indoor_exercise_day",
"mixed_environment_day"],
    "keh_plus_teg": ["indoor_exercise_day", "mixed_commute_day",
"outdoor_exercise_day"],
    "rest_only": ["sleep_heavy", "indoor_work_day", "outdoor_rest_day"],
    "solar_only": ["outdoor_rest_day", "cloudy_outdoor_day"],
    "active_commute": ["mixed_commute_day", "outdoor_exercise_day"],
}

def _resolve_tip_mass(tip_mass_g: float | None) -> float:
    if tip_mass_g is not None:
        return float(tip_mass_g)
    return float(PiezoModel().recommended_tip_mass_g()[0])

def load_trace(path: Path) -> pd.DataFrame:
    df = pd.read_csv(path, parse_dates=["time"])
    return df.sort_values("time").reset_index(drop=True)
```

```

def infer_dt_s(df: pd.DataFrame) -> float:
    if len(df) > 1:
        return float((df["time"].iloc[1] -
df["time"].iloc[0]).total_seconds())
    return 30.0

def combine_traces(dataset_dir: Path, scenario_names: List[str]) ->
pd.DataFrame:
    frames = []
    for day_idx, name in enumerate(scenario_names):
        f = dataset_dir / f"{name}.csv"
        if not f.exists():
            raise FileNotFoundError(f)
        df = load_trace(f).copy()
        df["time"] = df["time"] + pd.Timedelta(days=day_idx)
        frames.append(df)
    return pd.concat(frames, axis=0, ignore_index=True)

def recompute_harvest(df: pd.DataFrame, tip_mass_g: float | None = None)
-> pd.DataFrame:
    tip_mass_g = _resolve_tip_mass(tip_mass_g)
    teg = TEGModel()
    pv = PVModel()
    piezo = PiezoModel()
    out = df.copy()
    dt_s = infer_dt_s(out)
    out["teg_power_w"] = teg.output_power_w(out["skin_temp_c"].to_numpy(),
out["ambient_temp_c"].to_numpy())
    pv_irr = np.where(out["light_lux"].to_numpy() > 350.0,
out["irradiance_wm2"].to_numpy(), 0.0)
    out["pv_power_w"] = pv.irradiance_to_power_w(pv_irr,
out["incidence_deg"].to_numpy(), out["ambient_temp_c"].to_numpy() + 3.0,
out["light_lux"].to_numpy())

```

```

    out["keh_power_w"] = piezo.power_w(out["activity"].to_numpy(),
out["step_rate_hz"].to_numpy(), out["accel_rms_g"].to_numpy(), dt_s=1.0,
tip_mass_g=tip_mass_g)
    out["harvest_total_w"] = out[["teg_power_w", "pv_power_w",
"keh_power_w"]].sum(axis=1)
    out["harvest_total_j"] = out["harvest_total_w"] * dt_s
    out["tip_mass_g"] = tip_mass_g
    return out

```

```

def run_scheduler_on_combined(df: pd.DataFrame) -> pd.DataFrame:
    buffer = HybridBuffer()
    sched = WearableScheduler()
    buffer.reset(sc_v=2.35, bat_frac=0.42)
    dt_s = infer_dt_s(df)
    roll_window = max(1, int(round(300.0 / max(dt_s, 1e-9))))
    rolling_pred = df["harvest_total_w"].rolling(roll_window,
min_periods=1).mean().to_numpy()
    pending_samples = 0
    last_sample_t = -10**9
    rows = []

    for t_idx, row in df.iterrows():
        harvest_w = float(row["harvest_total_w"])
        harvest_j = harvest_w * dt_s
        buffer.charge(harvest_j)
        predicted = float(rolling_pred[t_idx])
        step = sched.step(buffer, t_idx, row, predicted, last_sample_t,
pending_samples, tick_s=dt_s)
        pending_samples = int(step["pending_samples"])
        last_sample_t = int(step["last_sample_t"])
        rows.append(
            {
                "time": row["time"],
                "scenario": row["scenario"],
                "activity": row["activity"],
                "harvest_total_w": harvest_w,
                "harvest_total_j": harvest_j,
                "teg_power_w": float(row["teg_power_w"]),

```

```

        "pv_power_w": float(row["pv_power_w"]),
        "keh_power_w": float(row["keh_power_w"]),
        "sc_v": step["sc_v"],
        "bat_v": step["bat_v"],
        "pending_samples": pending_samples,
        "do_sample": int(step["do_sample"]),
        "do_tx": int(step["do_tx"]),
        "period_s": int(step["period_s"]),
        "total_energy_j": buffer.total_energy_j(),
    }

    )

    return pd.DataFrame(rows)

def build_and_save(dataset_dir: Path, out_dir: Path, tip_mass_g: float |
None = None) -> Dict[str, Path]:
    out_dir.mkdir(parents=True, exist_ok=True)
    tip_mass_g = _resolve_tip_mass(tip_mass_g)
    outputs = {}
    for scenario_name, parts in SCENARIO_MAP.items():
        combined = combine_traces(dataset_dir, parts)
        combined = recompute_harvest(combined, tip_mass_g=tip_mass_g)
        combined_path = out_dir /
f"scenario_combined_timeseries_{scenario_name}.csv"
        combined.to_csv(combined_path, index=False)
        sched_df = run_scheduler_on_combined(combined)
        sched_path = out_dir / f"scheduler_trace_{scenario_name}.csv"
        sched_df.to_csv(sched_path, index=False)
        dt_s = infer_dt_s(combined)
        metrics = {
            "scenario": scenario_name,
            "rows": int(len(combined)),
            "duration_h": float(len(combined) * dt_s / 3600.0),
            "sample_period_s": dt_s,
            "mean_total_harvest_mw":
float(combined["harvest_total_w"].mean() * 1000.0),
            "harvest_j": float(combined["harvest_total_w"].sum() * dt_s),
            "samples": int(sched_df["do_sample"].sum()),
            "tx": int(sched_df["do_tx"].sum()),
            "final_sc_v": float(sched_df["sc_v"].iloc[-1]),

```

```

        "final_bat_v": float(sched_df["bat_v"].iloc[-1]),
        "tip_mass_g": tip_mass_g,
    }
    (out_dir /
f"summary_{scenario_name}.json").write_text(json.dumps(metrics, indent=2))
    outputs[scenario_name] = combined_path
    print(f"built {scenario_name}: {metrics['samples']} samples,
{metrics['tx']} tx")
    return outputs

def main() -> None:
    parser = argparse.ArgumentParser()
    parser.add_argument("--dataset-dir", type=Path,
default=Path("../dataset"))
    parser.add_argument("--out-dir", type=Path,
default=Path("../outputs"))
    parser.add_argument("--tip-mass-g", type=float, default=None,
help="Override the model-selected tip mass")
    args = parser.parse_args()
    build_and_save(args.dataset_dir.resolve(), args.out_dir.resolve(),
tip_mass_g=args.tip_mass_g)

if __name__ == "__main__":
    main()

```


simulator.py

```
#!/usr/bin/env python3
"""Compare scheduler policies on the combined wearable energy-harvesting
traces."""
from __future__ import annotations

from pathlib import Path
import argparse
import json
from typing import Dict, List

import numpy as np
import pandas as pd

from core_harvesters import HybridBuffer, WearableScheduler,
SchedulerConfig

POLICIES = ["periodic", "threshold", "predictive", "opportunistic",
"energy_neutral"]

def load_combined(path: Path) -> pd.DataFrame:
    df = pd.read_csv(path, parse_dates=["time"])
    return df.sort_values("time").reset_index(drop=True)

def infer_dt_s(df: pd.DataFrame) -> float:
    if len(df) > 1:
        return float((df["time"].iloc[1] -
df["time"].iloc[0]).total_seconds())
    return 30.0

def simulate_policy(df: pd.DataFrame, policy: str) -> pd.DataFrame:
    buffer = HybridBuffer()
    sched = WearableScheduler()
    buffer.reset(sc_v=2.30, bat_frac=0.40)

    cfg = SchedulerConfig()
```

```

pending_samples = 0
last_sample_t = -10**9
rows = []
dt_s = infer_dt_s(df)
roll_window = max(1, int(round(300.0 / max(dt_s, 1e-9))))
rolling_pred = df["harvest_total_w"].rolling(roll_window,
min_periods=1).mean().to_numpy()

for t_idx, row in df.iterrows():
    harvest_w = float(row["harvest_total_w"])
    harvest_j = harvest_w * dt_s
    buffer.charge(harvest_j)
    sc_v = buffer.sc_voltage()
    bat_v = buffer.battery_voltage()
    activity = str(row["activity"])
    predicted_w = float(rolling_pred[t_idx])

    period_s = sched.decide_period_s(sc_v, bat_v, predicted_w,
activity)
    idle_need = cfg.idle_energy_w * dt_s
    buffer.discharge(idle_need)

    # Every policy still respects an explicit inter-sample interval.
    if policy == "periodic":
        target_period = cfg.base_sample_period_s
    elif policy == "threshold":
        target_period = cfg.high_energy_period_s if (sc_v >=
cfg.sc_high_threshold_v and bat_v >= cfg.bat_low_threshold_v) else
cfg.low_energy_period_s
    elif policy == "predictive":
        target_period = cfg.base_sample_period_s if predicted_w *
600.0 >= (cfg.sample_energy_j + cfg.process_energy_j +
cfg.tx_base_energy_j / cfg.batch_size) else cfg.low_energy_period_s
    elif policy == "opportunistic":
        target_period = cfg.high_energy_period_s if harvest_w >=
0.0008 else cfg.low_energy_period_s
    elif policy == "energy_neutral":
        target_period = period_s
    else:
        target_period = cfg.base_sample_period_s

```

```

do_sample = False
do_tx = False
if (t_idx - last_sample_t) >= max(1, int(np.ceil(target_period /
max(dt_s, 1e-9)))):
    if policy != "opportunistic" or harvest_w >= 0.0008:
        need = cfg.wake_energy_j + cfg.sample_energy_j +
cfg.process_energy_j
        used = buffer.discharge(need)
        if used["unmet_j"] <= 0:
            do_sample = True
            pending_samples += 1
            last_sample_t = t_idx

    # Transmission policy: send in batches and prefer times of surplus
energy.
    if pending_samples >= cfg.batch_size:
        tx_need = cfg.tx_base_energy_j + pending_samples *
cfg.tx_per_sample_energy_j
        if sc_v >= cfg.sc_high_threshold_v or predicted_w > 0.0010:
            used = buffer.discharge(tx_need)
            if used["unmet_j"] <= 0:
                do_tx = True
                pending_samples = 0

    # tiny emergency telemetry when energy is abundant and motion is
high.
    if activity in {"outdoor_exercise", "outdoor_walk", "commute"} and
float(row["accel_rms_g"]) > 1.6 and sc_v >= cfg.sc_high_threshold_v:
        used = buffer.discharge(cfg.emergency_event_energy_j)
        if used["unmet_j"] <= 0:
            do_tx = True

rows.append({
    "time": row["time"],
    "policy": policy,
    "scenario": row["scenario"],
    "activity": activity,
    "harvest_total_w": harvest_w,
    "harvest_total_j": harvest_j,

```

```

        "sc_v": sc_v,
        "bat_v": bat_v,
        "pending_samples": pending_samples,
        "do_sample": int(do_sample),
        "do_tx": int(do_tx),
        "total_energy_j": buffer.total_energy_j(),
        "period_s": period_s,
    })
    return pd.DataFrame(rows)

def summarize(out: pd.DataFrame) -> Dict[str, float]:
    dt_s = infer_dt_s(out)
    duration_s = len(out) * dt_s
    return {
        "policy": out["policy"].iloc[0],
        "duration_h": duration_s / 3600.0,
        "samples": int(out["do_sample"].sum()),
        "tx": int(out["do_tx"].sum()),
        "mean_sc_v": float(out["sc_v"].mean()),
        "final_sc_v": float(out["sc_v"].iloc[-1]),
        "final_bat_v": float(out["bat_v"].iloc[-1]),
        "mean_total_harvest_mw": float(out["harvest_total_w"].mean() *
1000.0),
        "harvest_j": float(out["harvest_total_j"].sum()),
        "uptime_fraction": float(out["do_sample"].mean()),
    }

def main() -> None:
    parser = argparse.ArgumentParser()
    parser.add_argument("--combined", type=Path, required=True,
help="scenario_combined_timeseries_*.csv")
    parser.add_argument("--out-dir", type=Path,
default=Path("../outputs"))
    args = parser.parse_args()
    df = load_combined(args.combined)
    args.out_dir.mkdir(parents=True, exist_ok=True)

    summaries = []

```

```

for pol in POLICIES:
    out = simulate_policy(df, pol)
    out_path = args.out_dir / f"policy_{pol}_results.csv"
    out.to_csv(out_path, index=False)
    summaries.append(summarize(out))
    print(f"saved {out_path.name}")

    pd.DataFrame(summaries).to_csv(args.out_dir / "policy_summary.csv",
index=False)
    (args.out_dir /
"policy_summary.json").write_text(json.dumps(summaries, indent=2))

if __name__ == "__main__":
    main()

```

References

- [1] C. Xu *et al.*, “Portable and wearable self-powered systems based on emerging energy harvesting technology,” *Microsystems & Nanoengineering*, vol. 7, article 25, 2021. doi:10.1038/s41378-021-00248-z.
- [2] M. Kang and W.-H. Yeo, “Advances in energy harvesting technologies for wearable devices,” *Micromachines*, vol. 15, no. 7, p. 884, 2024. doi:10.3390/mi15070884.
- [3] S. Ahmed *et al.*, “The Internet of Batteryless Things,” *Commun. ACM*, vol. 68, no. 2, pp. 28–37, Feb. 2024.
- [4] H. Liu, H. Fu, L. Sun, C. Lee, and E. Y. Yeatman, “Hybrid Energy Harvesting Technology: From Materials, Structural Design, System Integration to Applications,” *Renew. Sust. Energy Rev.*, vol. 137, p. 110473, 2021. doi:10.1016/j.rser.2021.110473.
- [5] A. Simone, P. Marino, R. Greco, and A. Lo Schiavo, “Quantifying the Benefits of Hybrid Energy Harvesting from Natural Sources,” *Electronics*, vol. 14, no. 7, p. 1400, 2025. doi:10.3390/electronics14071400.
- [6] S. Saifi *et al.*, “An ultraflexible energy harvesting-storage system for wearable applications,” *Nat. Commun.*, vol. 15, article 6546, 2024. doi:10.1038/s41467-024-50894-w.
- [7] S. Mosavat, J. Göpfert, B.-C. Renner, and M. Zimmerling, “CATO: Wake-Up reCeiver-based communicaTiOn for Batteryless Devices,” *Sensors (Basel)*, vol. 25, no. 2, p. 582, 2025.
- [8] C. J. Lukas *et al.*, “A 1.02 μ W Battery-Less, Continuous Sensing and Post-Processing SiP for Wearable Applications,” *IEEE Trans. Biomed. Circuits Syst.*, vol. 13, no. 2, pp. 271–281, Apr. 2019. doi:10.1109/TBCAS.2019.2894775.
- [9] J. Yuan and R. Zhu, “A fully self-powered wearable monitoring system with systematically optimized flexible thermoelectric generator,” *Appl. Energy*, vol. 271, p. 115250, 2020. doi:10.1016/j.apenergy.2020.115250.
- [10] Y. Wang *et al.*, “Self-Powered Wearable Pressure Sensing System for Continuous Healthcare Monitoring Enabled by Flexible Thin-Film Thermoelectric Generator,” *Nano Energy*, vol. 73, p. 104773, 2020. doi:10.1016/j.nanoen.2020.104773.
- [11] Y. Wang *et al.*, “Hierarchically patterned self-powered sensors for multifunctional tactile sensing,” *Sci. Adv.*, vol. 6, no. eabb9083, 2020. doi:10.1126/sciadv.abb9083.
- [12] X. Zhao *et al.*, “Hybridized Nanogenerators for Effectively Scavenging Mechanical and Solar Energies,” *iScience*, vol. 24, p. 102415, 2021. doi:10.1016/j.isci.2021.102415.
- [13] H. Jinno *et al.*, “Stretchable and Waterproof Organic Photovoltaics for Washable Electronic Textile Applications,” *Nat. Energy*, vol. 2, pp. 780–785, 2017. doi:10.1038/s41560-017-0009-8.
- [14] J. Min *et al.*, “An Autonomous Wearable Biosensor Powered by a Perovskite Solar Cell,” *Nat. Electron.*, vol. 6, pp. 630–641, 2023. doi:10.1038/s41928-023-01010-7.

- [15] B. J. Restrepo-Cuestas, A. Ortiz-Pasos, and M. Durango-Flórez, "Electrical behavior dataset of photovoltaic mini panels under indoor and outdoor conditions," *Data in Brief*, vol. 60, p. 111607, Jun. 2025, doi: <https://doi.org/10.1016/j.dib.2025.111607>.
- [16] A. Hongn, F. Bosch, L. E. Prado, José Manuel Ferrández, and M. P. Bonomini, "Wearable Physiological Signals under Acute Stress and Exercise Conditions," *Scientific Data*, vol. 12, no. 1, Mar. 2025, doi: <https://doi.org/10.1038/s41597-025-04845-9>.
- [17] G. Weiss. "WISDM Smartphone and Smartwatch Activity and Biometrics Dataset ," UCI Machine Learning Repository, 2019. [Online]. Available: <https://doi.org/10.24432/C5HK59>.